DDD SAMPLE TUTORIAL

A SAMPLE DDD TUTORIAL IS GIVEN HERE TO SHOW YOU SOME OF THE MAIN FEATURES IN DDD AND SHOW YOU HOW TO USE DDD TO EXECUTE INSTRUCTIONS IN SINGLE STEP MODE (EXECUTING ONE INSTRUCTION AT A TIME) AND TO VIEW THE REGISTERS AND MEMORY CONTENTS.

PROGRAM USED IN THIS TUTORIAL IS sample1.as WHICH IS GIVEN BELOW

sample1.as

```
        .global _start

        .data
list:   .space 40

        .text
_start: movq $0x9876abcdef543210, %rax
        movq $list, %rsi
        movq %rax, (%rsi)
        movq (%rsi), %rdx
        addq %rax, %rdx
        pushq %rax
        pushq %rdx
        popq  %rax
        popq  %rdx

        movq $60, %rax
        xorq %rdi, %rdi
        syscall
```

ASSEMBLE, LINK AND CREATE EXECUTABLE OBJECT FILE

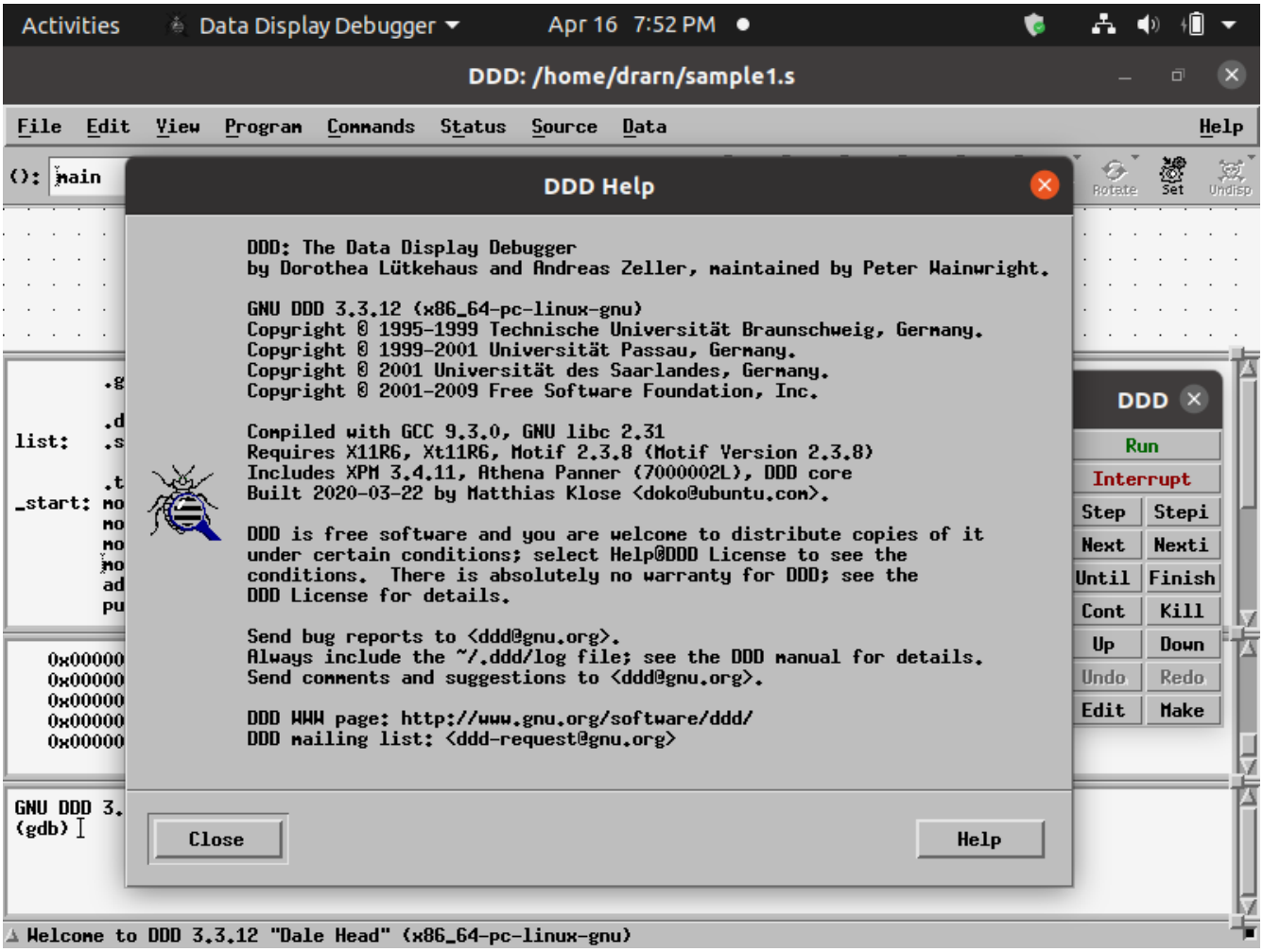
$as -gstabs -o sample1.o sample1.s

$ld -o sample1 sample1.o

INVOKE DDD DEBUGGER USING THE FOLLOWING COMMAND TO DEBUG sample1

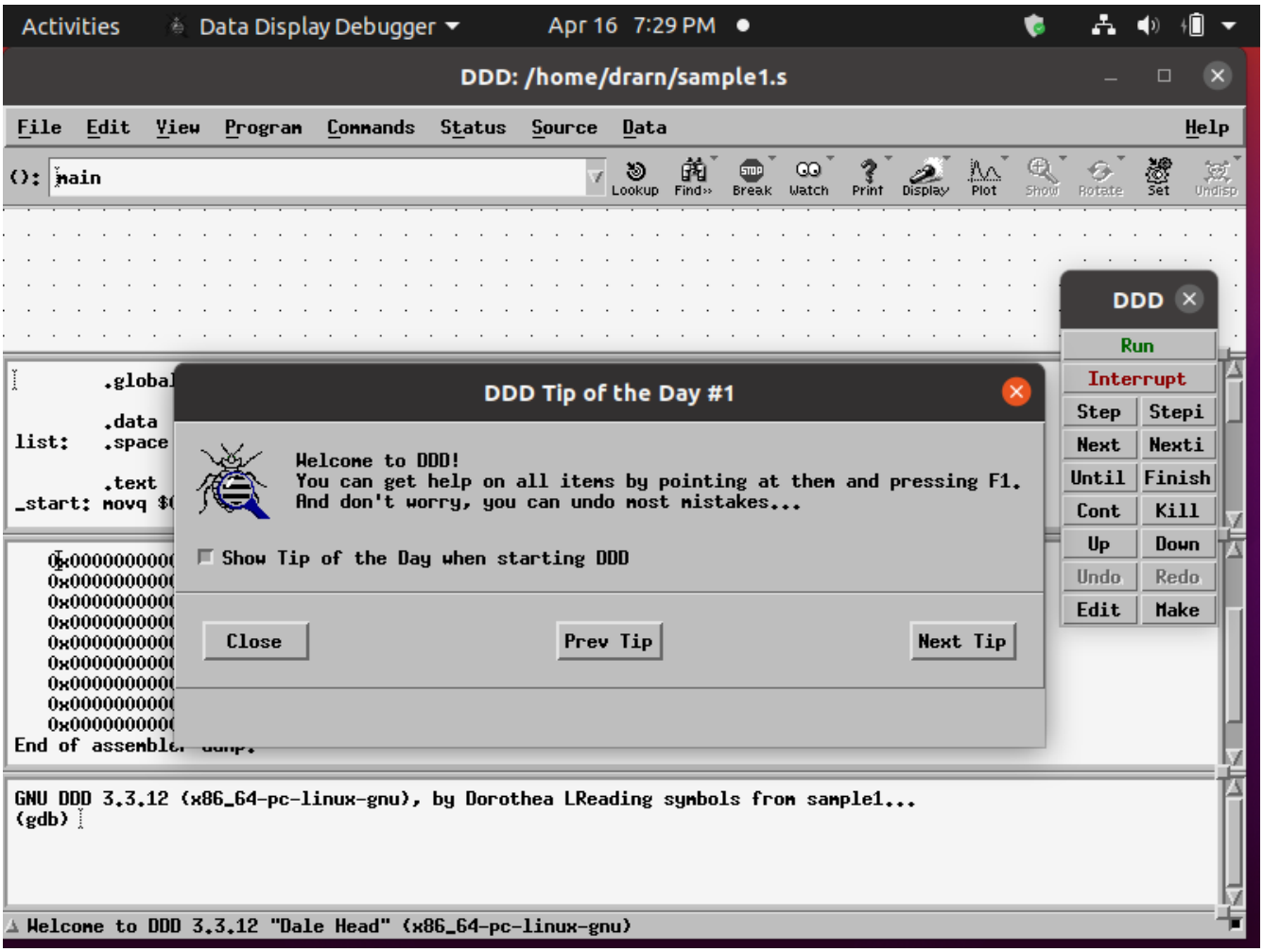
$ddd sample1&

DDD DEBUGGER OPENING WINDOW SHOWING DDD Help



CLOSE DDD HELP

DDD DEBUGGER WINDOW SHOWING DDD Tip of the Day #1



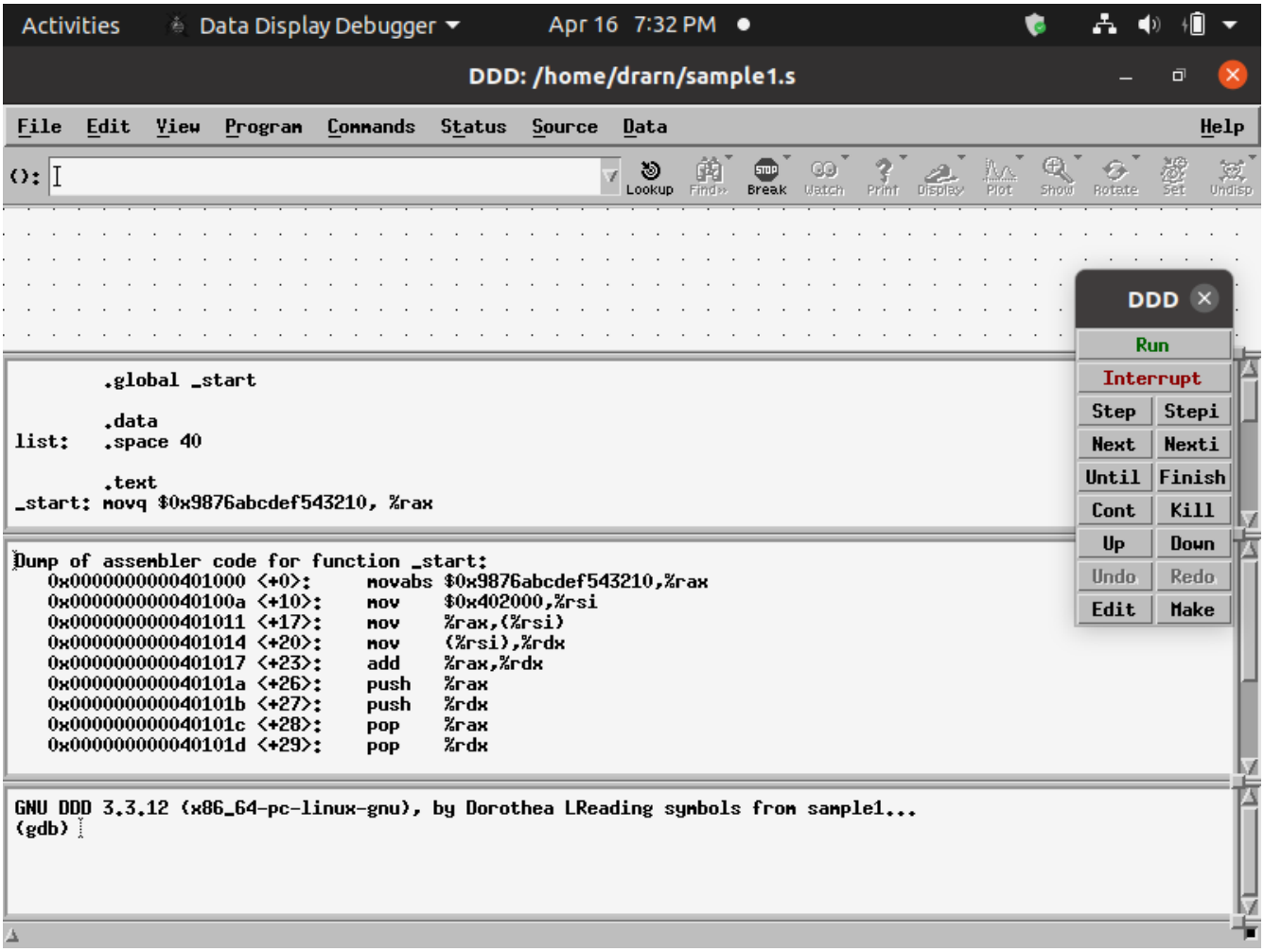
YOU CAN GO THROUGH ALL THE TIPS BY CLICKING NEXT TIP BUTTON

Close DDD Tip of the Day

NOW LOOK AT THE DDD DEBUGGER MENU OPTIONS

File Edit View Program Commands Status Source Data

CLICK ON EACH MENU OPTION TO KNOW THE SUB OPTIONS UNDER EACH MENU OPTION



ALSO LOOK AT THE DDD DEBUGGER SCREEN WINDOWS

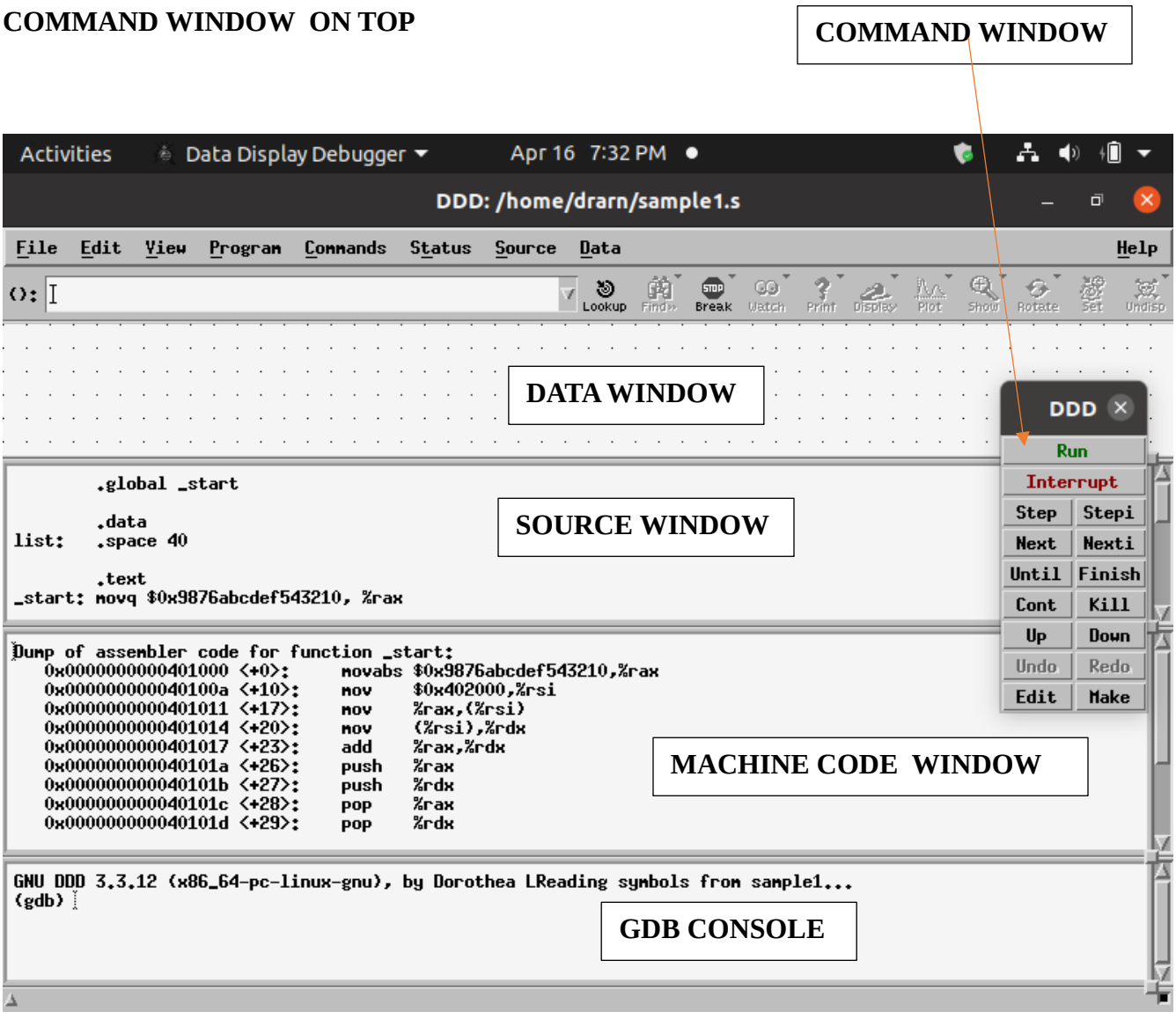
DATA WINDOW

SOURCE WINDOW

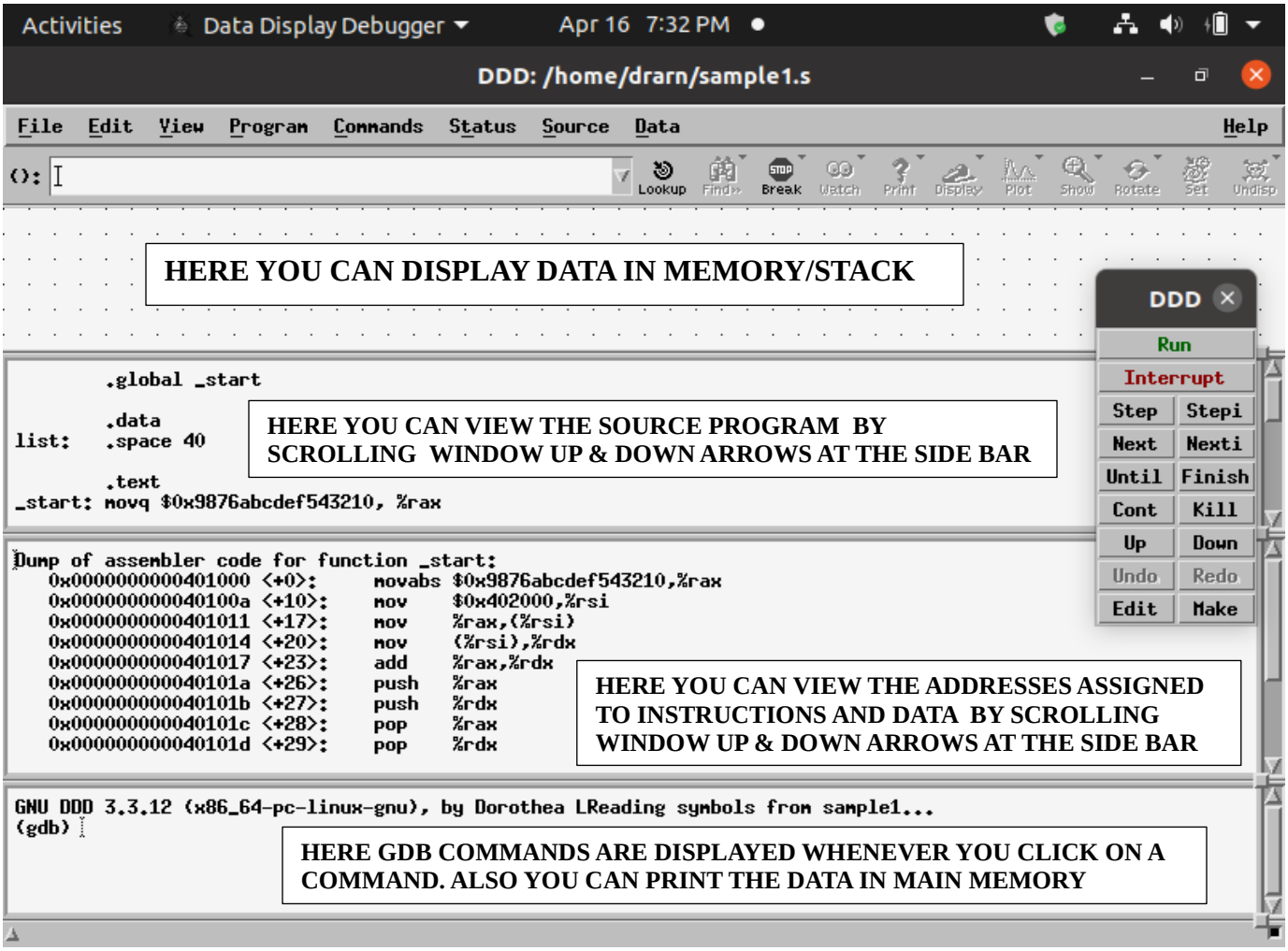
MACHINE CODE WINDOW

GDB CONSOLE

COMMAND WINDOW ON TOP



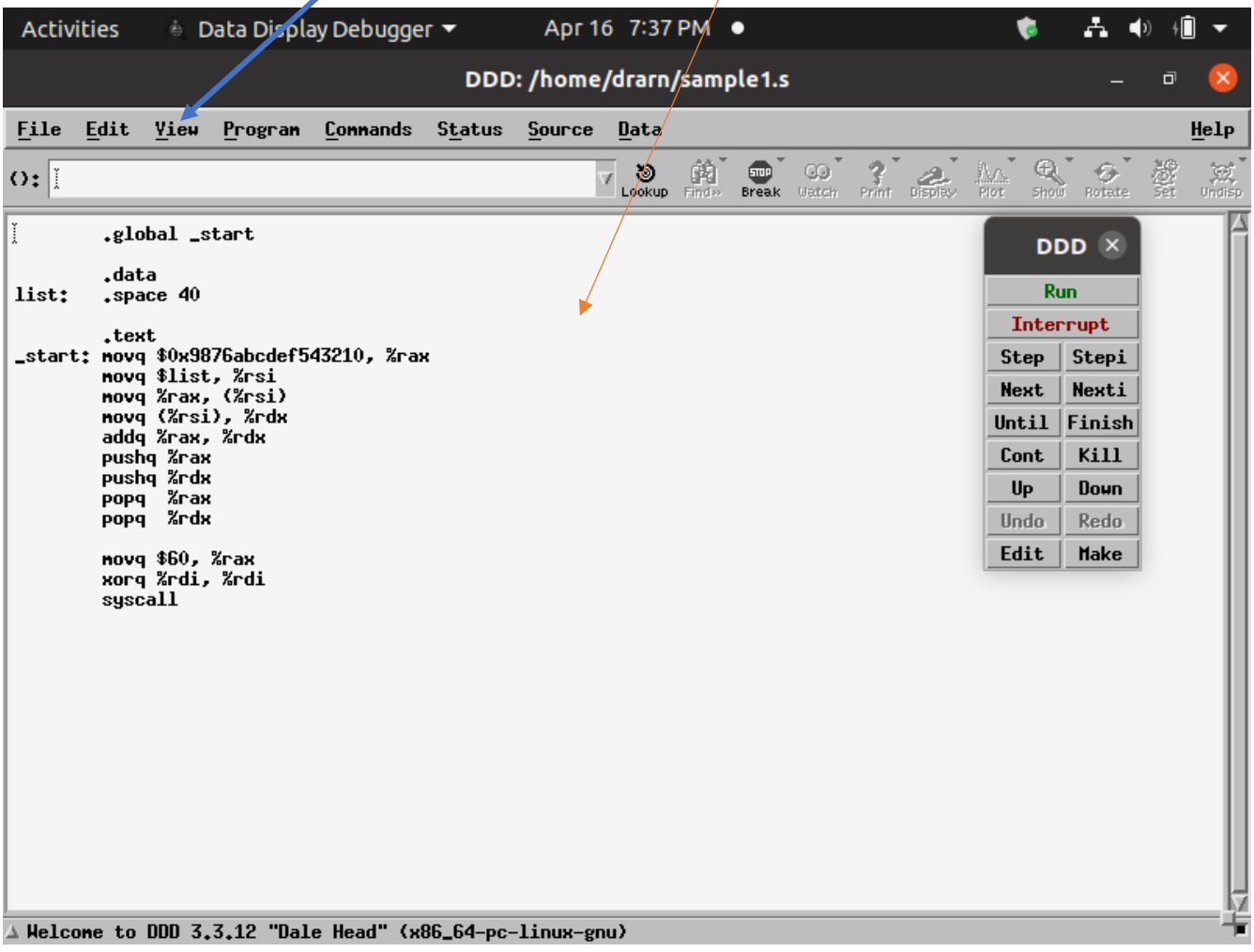
DDD DEBUGGER WINDOWS



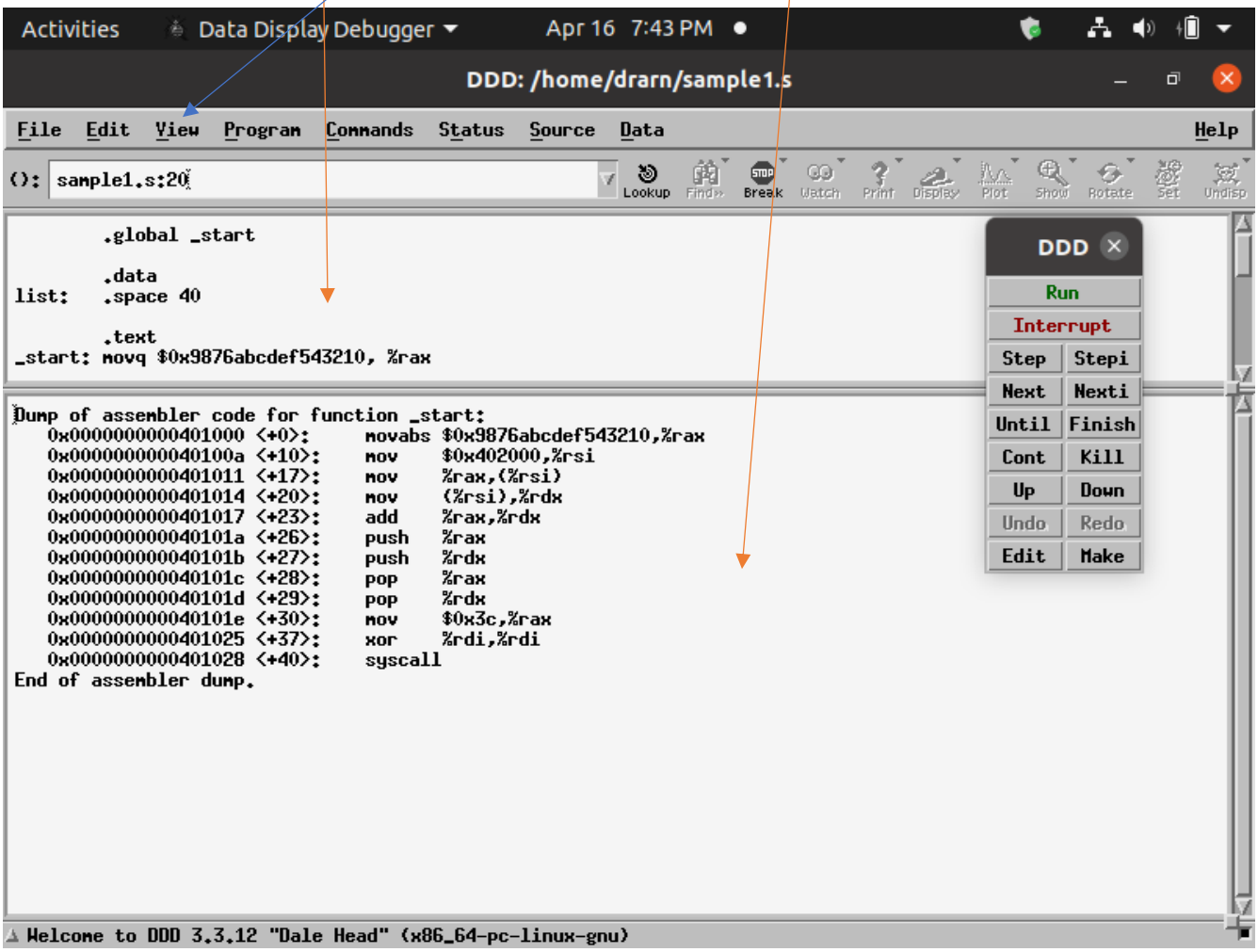
TO ENABLE OR DISABLE WINDOWS, CLICK ON View MENU OPTION TO SEE THE WINDOWS OPTIONS

- **DATA WINDOW**
- **SORCE WINDOW**
- **MACHINE CODE WINDOW**
- **GDB CONSOLE**

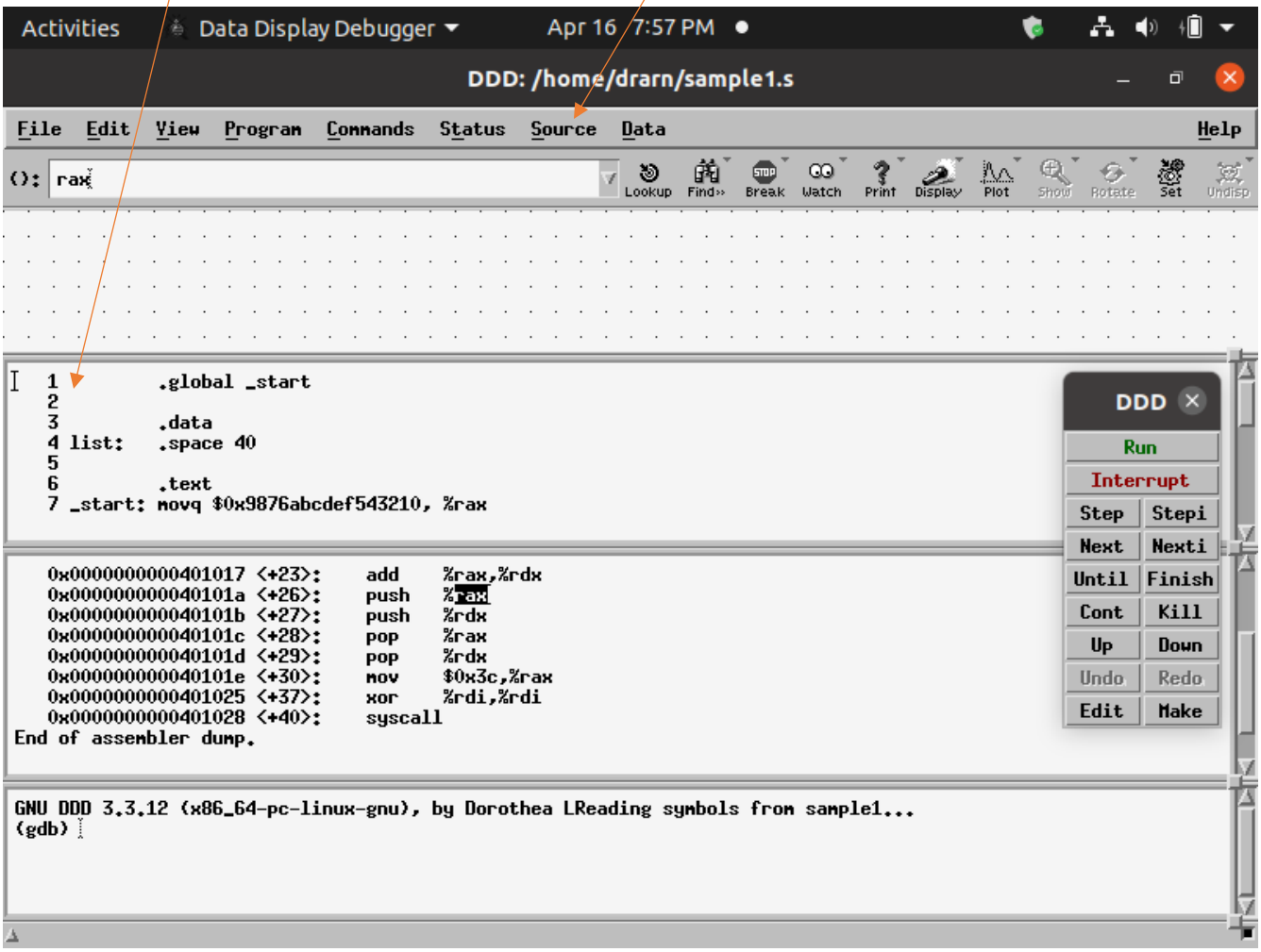
BY CLICKING ON THESE OPTIONS YOU CAN ENABLE OR DISABLE SELECTIVELY DIFFERENT WINDOWS
AS SHOWN BELOW – TO DISPLAY ONLY SOURCE WINDOW, DISABLE ALL OTHER WINDOWS BY CLICKING ON THEIR OPTIONS



TO DISPLAY ONLY SOURCE WINDOW AND MACHINE CODE WINDOW, SELECT THEM BY ENABLING THE OPTIONS AND DISABLE ALL OTHER WINDOWS BY CLICKING ON THEIR OPTIONS UNDER VIEW SUB MENU

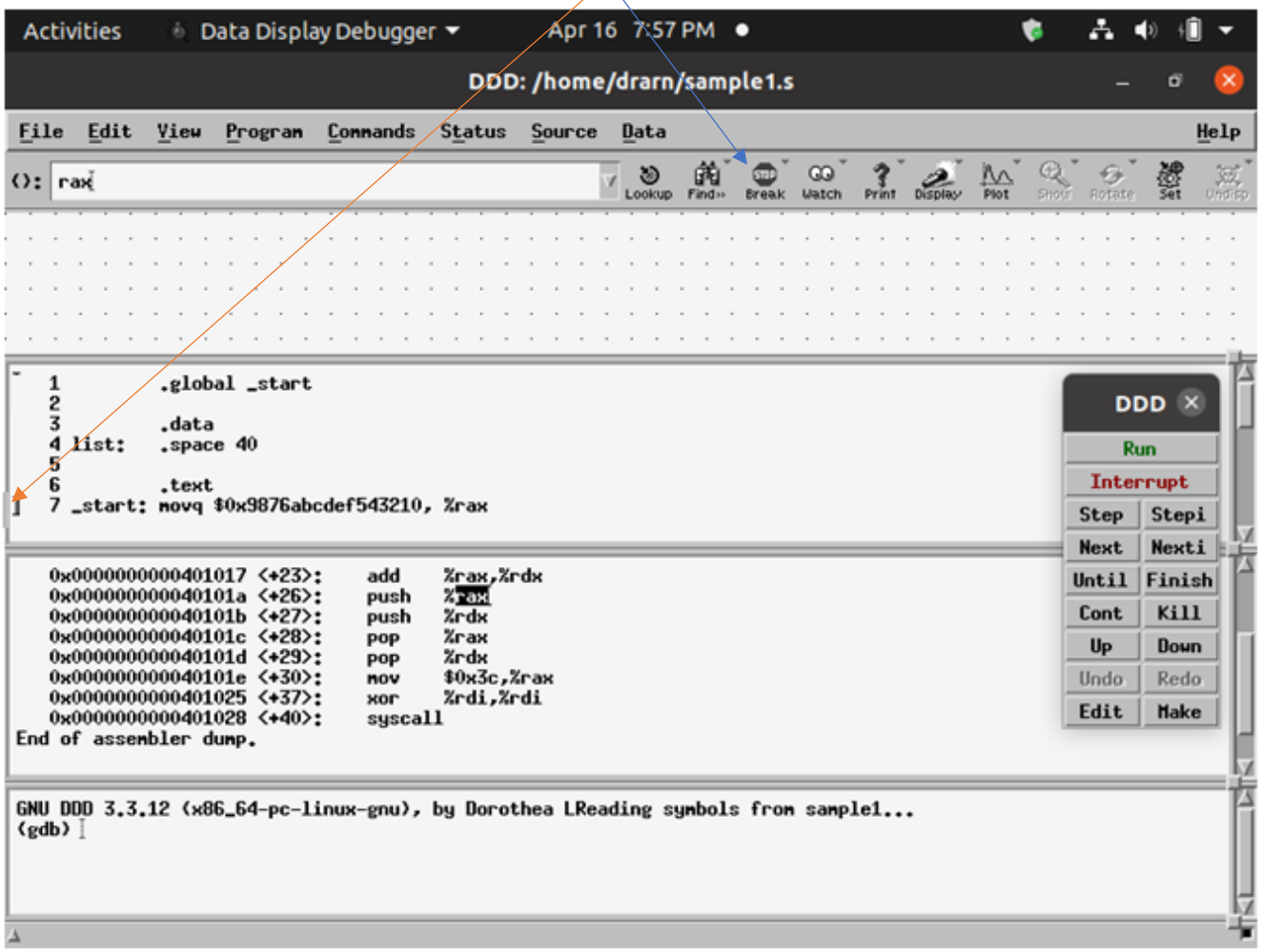


TO DISPLAY LINE NUMBERS, CLICK ON Source SUB MENU AND SELECT Display line numbers option

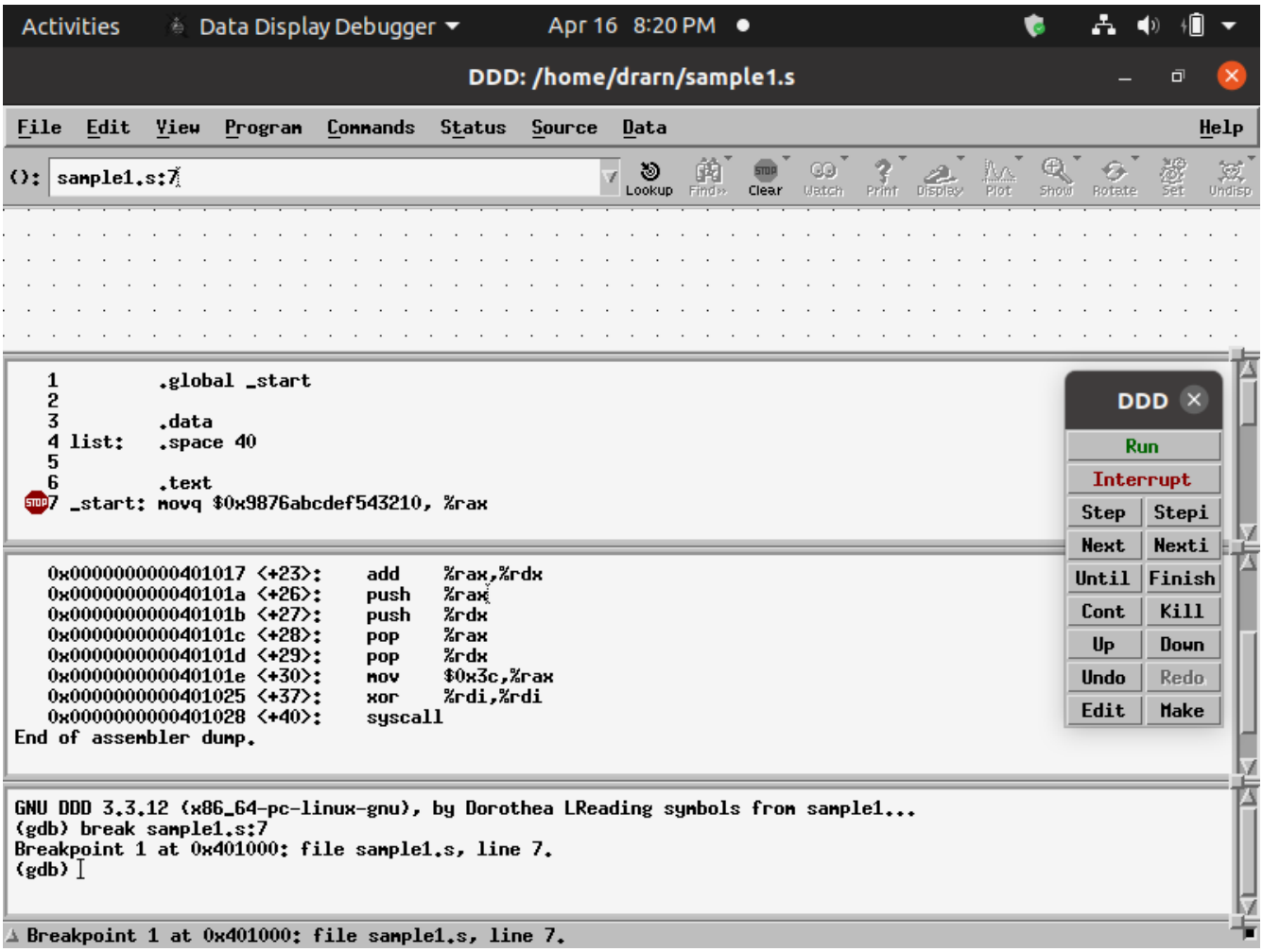


BEFORE START EXECUTING YOUR PROGRAM, PUT BREAKPOINT AT THE FIRST INSTRUCTION IN YOUR PROGRAM, THAT IS, INSTRUCTION AT LINE NO. 7 WITH THE LABEL _start:

IN ORDER TO DO THAT FIRST MOVE THE CURSOR TO LINE NO.7 AND KEEP CURSOR BEFORE LINE NO. 7 AND CLICK ON Break BUTTON

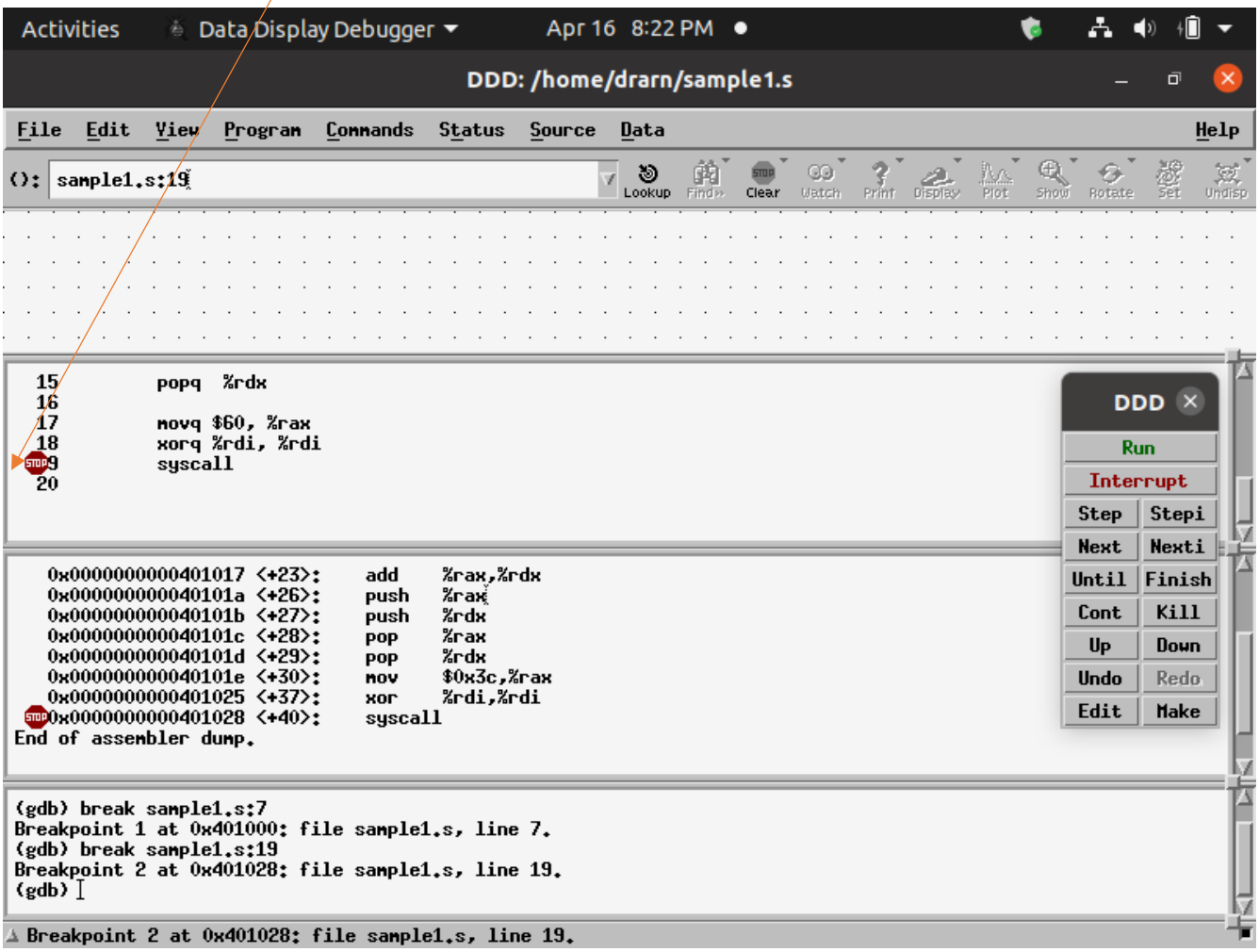


BREAKPOINT IS SET AT LINE NO. 7 – FIRST INSTRUCTION



THEN SET THE BREKPOINT AT THOSE INSTRUCTIONS LIKE syscall, call printf, call scanf, WHICH ARE TO BE EXECUTED USING RUN AND NOT USING SINGLE STEP

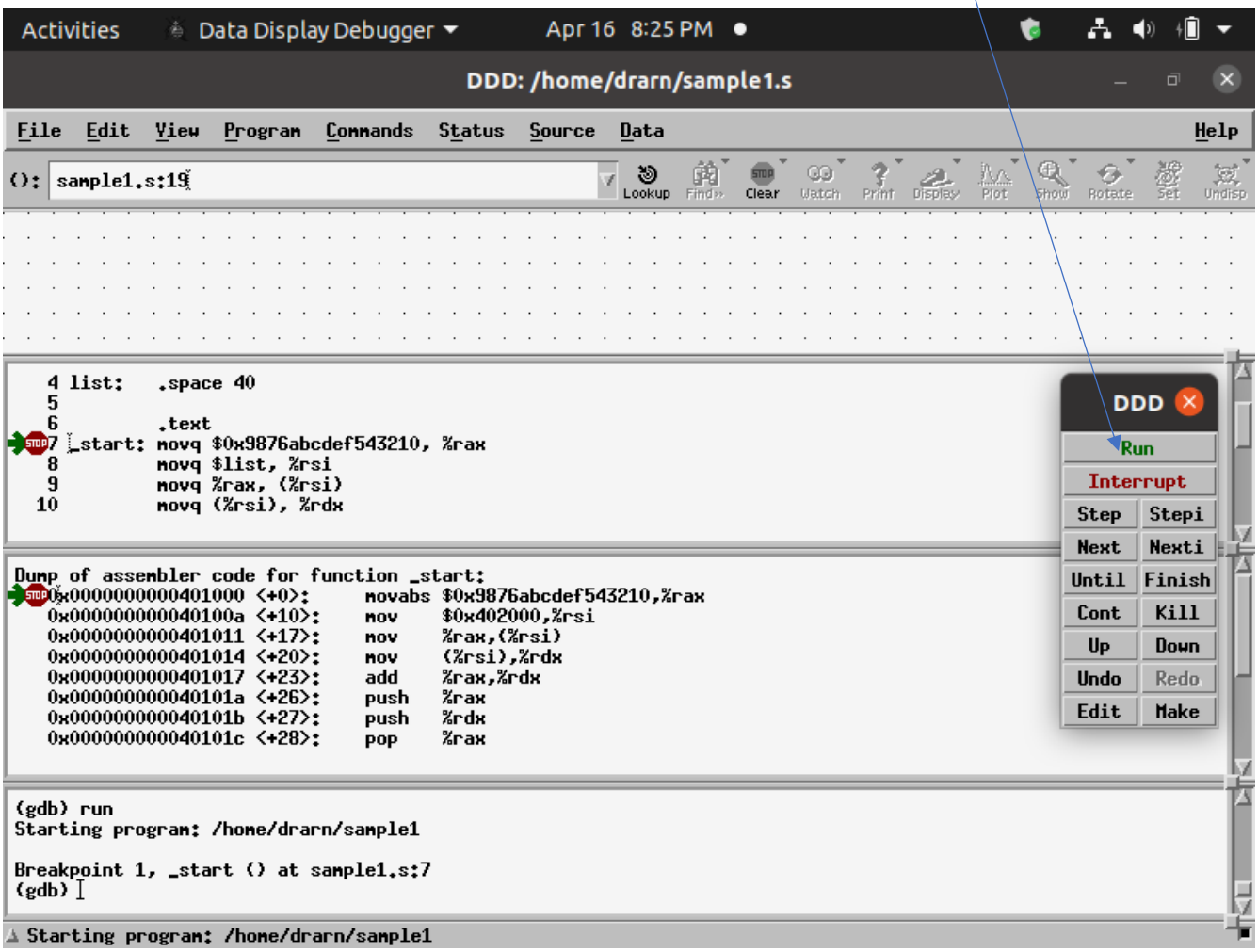
IN OUR PROGRAM, WE NEED TO SET BREAKPOINT AT LINE NO. 19, THAT IS, syscall INSTRUCTION AS SHOWN BELOW.



NOW EXECUTE THE PROGRAM BY CLICKING Run IN THE COMMAND WINDOW

SINCE THERE IS BREAKPOINT AT FIRST INSTRUCTION, EXECUTION WILL STOP OR PAUSE AT FIRST INSTRUCTION

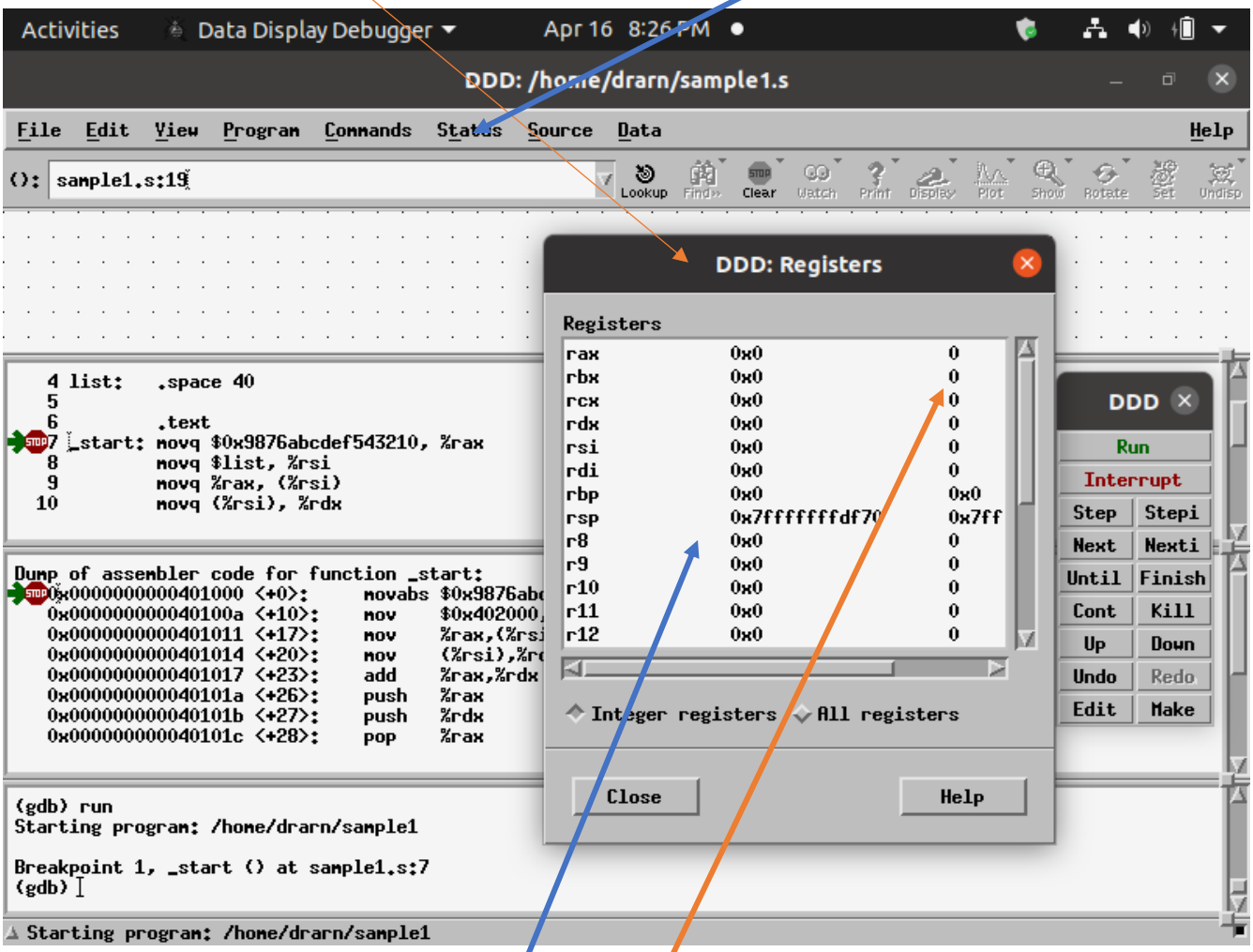
ALL GENERAL PURPOSE REGISTERS ARE CLEARED AND MEMORY LOCATIONS ARE CLEARED
%rip points to CODE SEGMENT AND %rsp POINTS TO TOP OF THE STACK



INITIALLY ALL GENERAL PURPOSE REGISTERS ARE CLEARED AND MEMORY LOCATIONS ARE CLEARED
%rip points to CODE SEGMENT AND %rsp POINTS TO TOP OF THE STACK

IN ORDER TO DISPLAY THE REGISTER WINDOW, CLICK OPTION Status AND SELECT THE SUB OPTION Registers UNDER THAT MENU

NOW REGISTER WINDOW IS POPPED UP. YOU CAN VIEW THE CONTENTS OF ALL THE REGISTERS IN HEX AND DECIMAL IN MOST OF THE CASES.



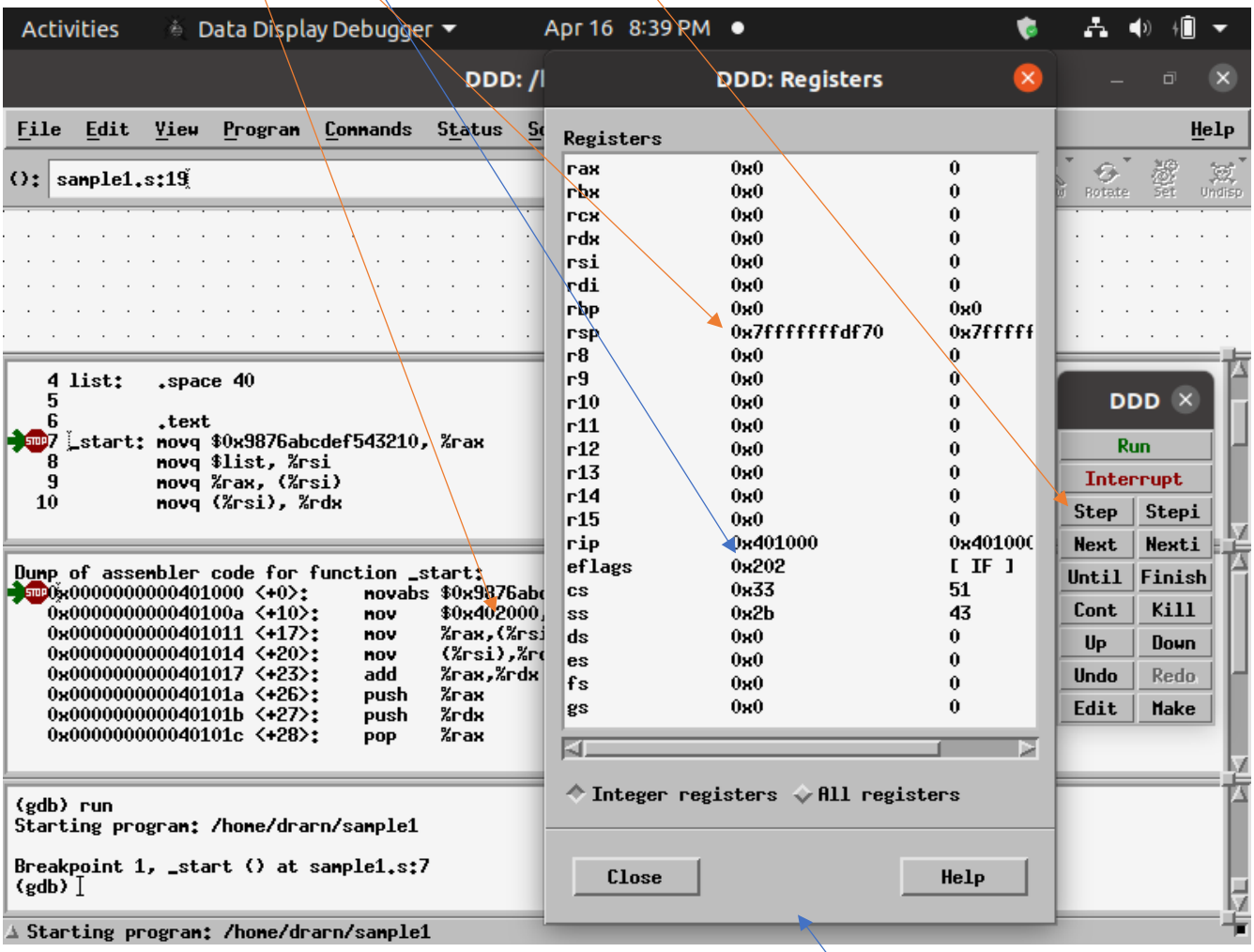
ALL REGISTER CONTENTS ARE SHOWN IN HEX
FOR GENERAL PURPOSE REGISTERS, REGISTER CONTENTS ARE ALSO SHOWN IN DECIMAL

%rip ← 0x401000 ;START OF CODE SEGMENT

list ← 0x402000 ;START OF DATA SEGMENT

%rsp ← 0x7fffffffdf70 ; TOP OF THE STACK

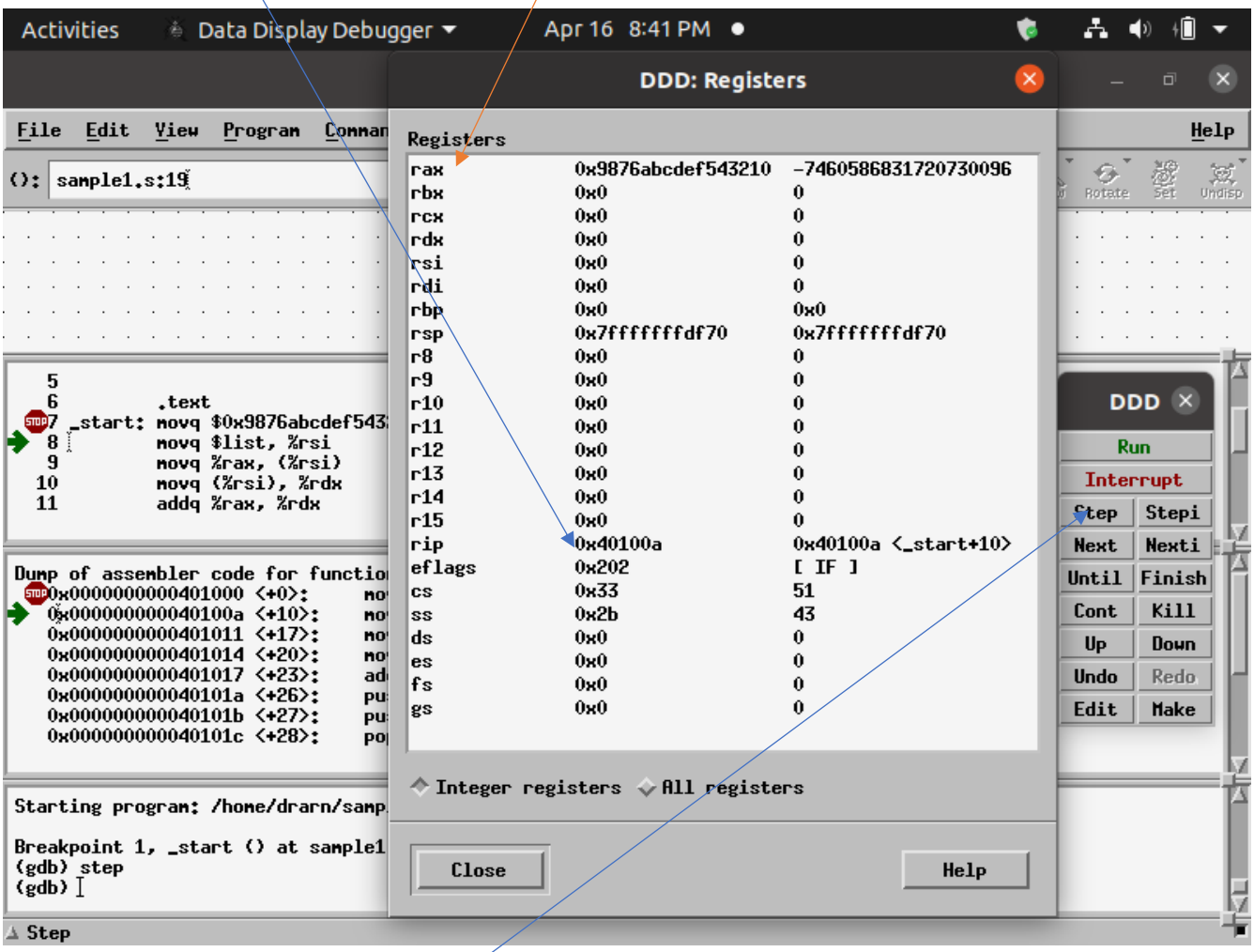
CLICK step to execute first Instruction at line no.7



ENLARGE THE REGISTER WINDOW BY PULLING IT DOWN

LOOK AT REGISTER WINDOW AFTER THE EXECUTION OF FIRST INSTRUCTION **movq $0x9876abcdef543210, %rax**

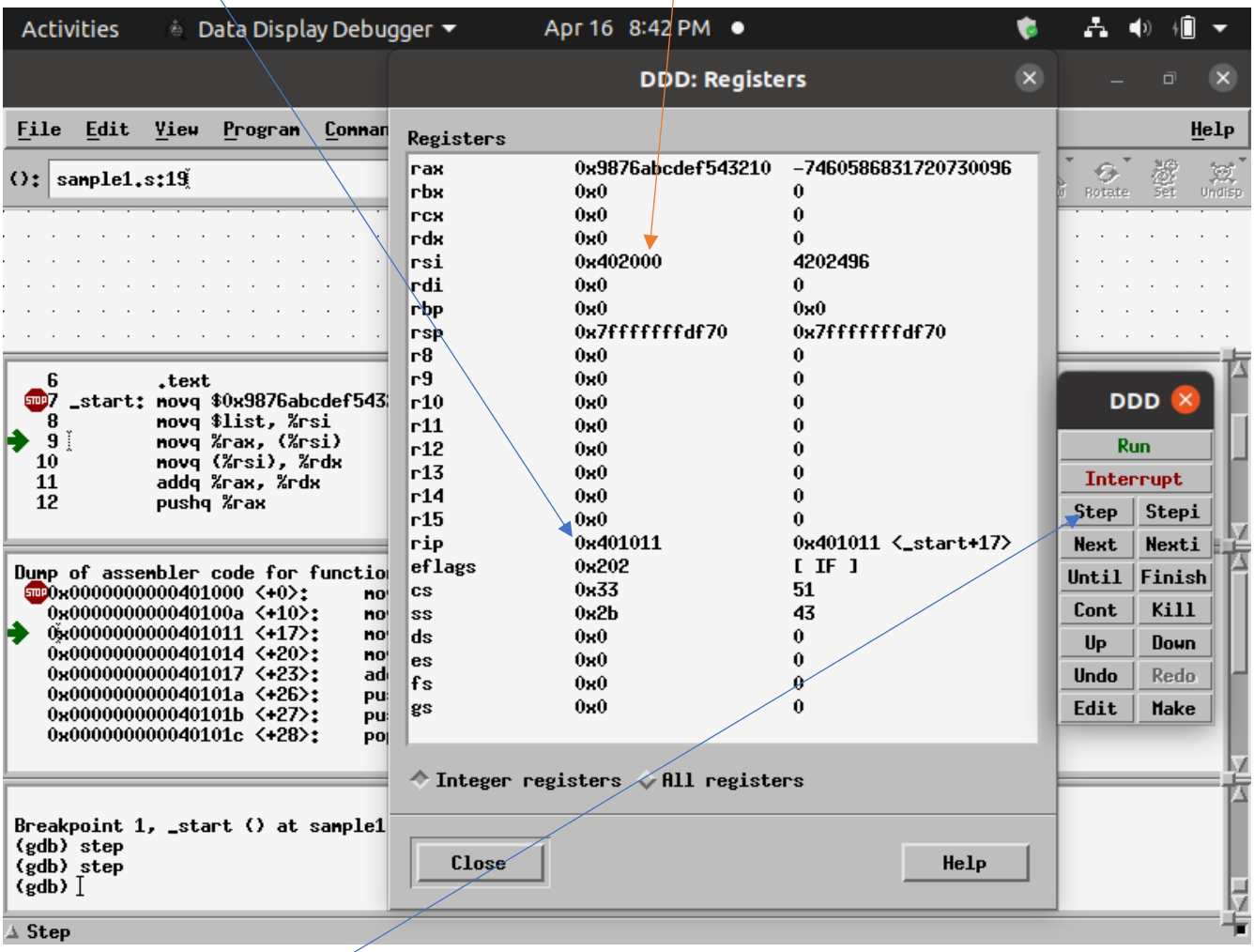
%rip ← 0x40100a ; ADDRESS OF SECOND INSTRUCTION



NOW CLICK step to execute second Instruction at line no.8 **movq $list, %rsi**

LOOK AT REGISTER WINDOW AFTER THE EXECUTION OF SECOND INSTRUCTION **movq $list, %rsi**

list ← 0x402000 ;START OF DATA BLOCK
%rip ← 0x401011 ; ADDRESS OF THIRD INSTRUCTION

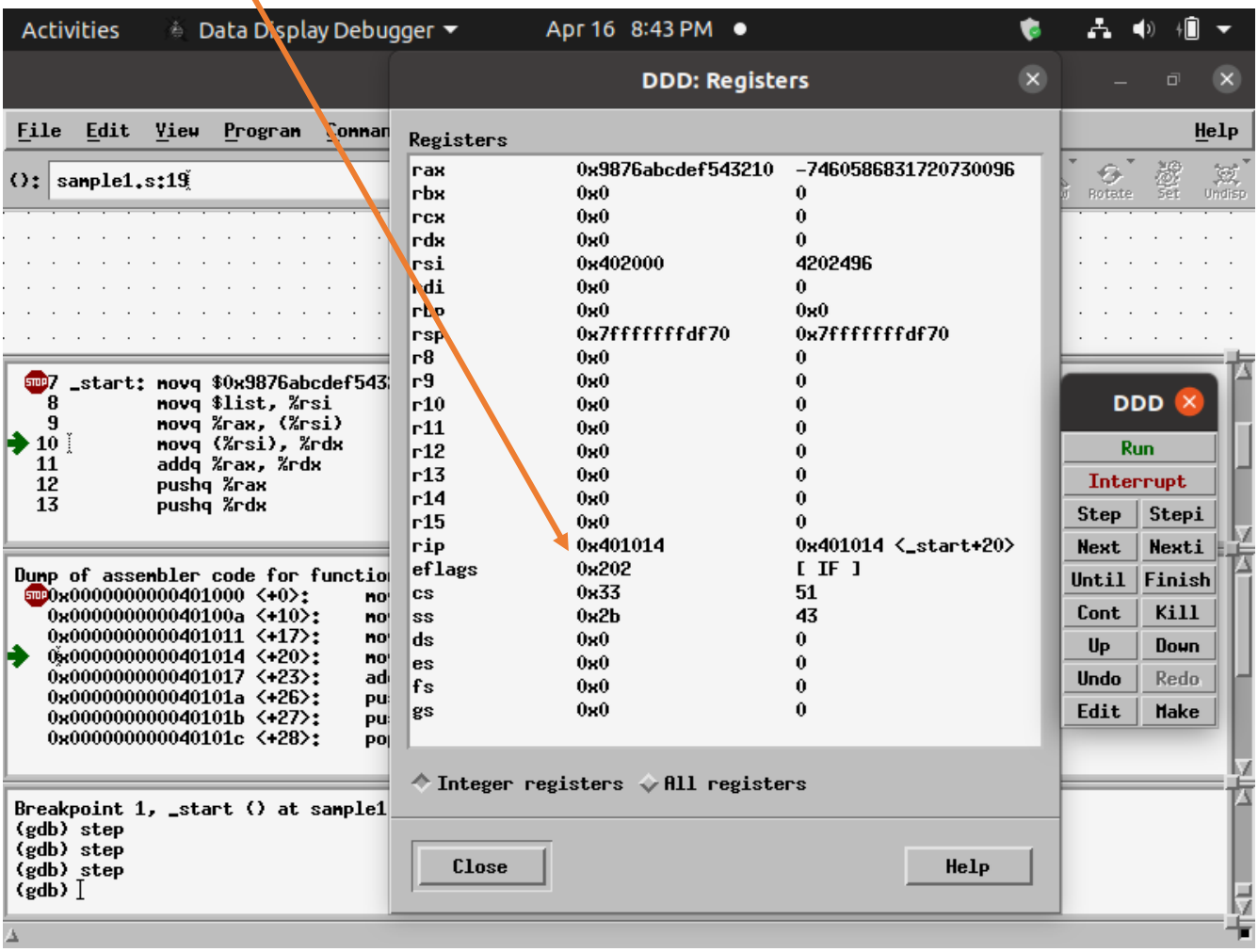


NOW CLICK step to execute THIRD Instruction at line no.9
movq %rax, (%rsi)

LOOK AT REGISTER WINDOW AFTER THE EXECUTION OF THIRD INSTRUCTION **movq %rax, (%rsi)**

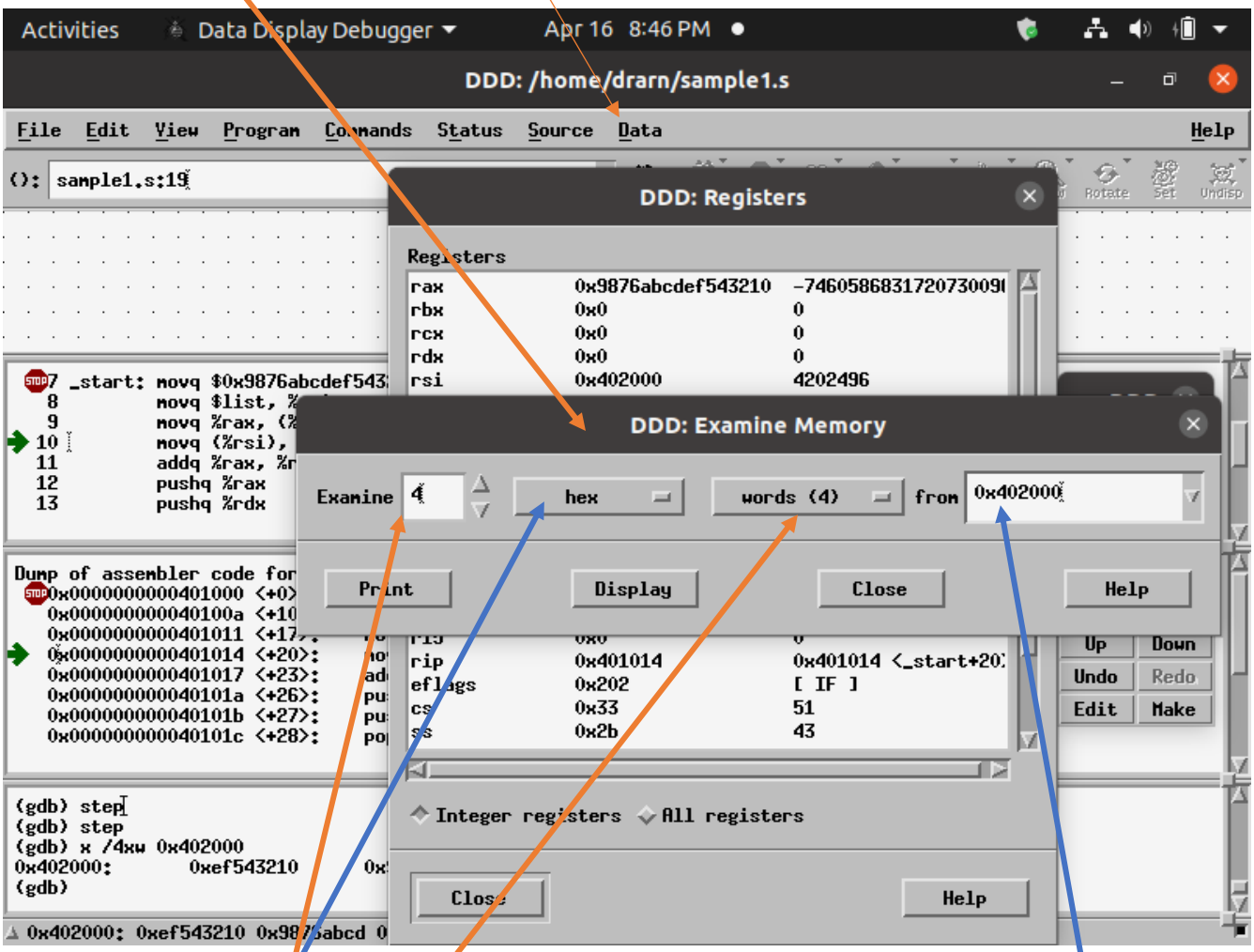
CONTENTS OF %rax REGISTER ARE WRITTEN INTO MEMORY STARTING FROM 0x402000

%rip ← 0x401014 ; ADDRESS OF FOURTH INSTRUCTION



LET US NOW LOOK AT MEMORY STARING AT 0x402000 where contents of %rax are written to memory area pointed to by %rsi = 0x402000

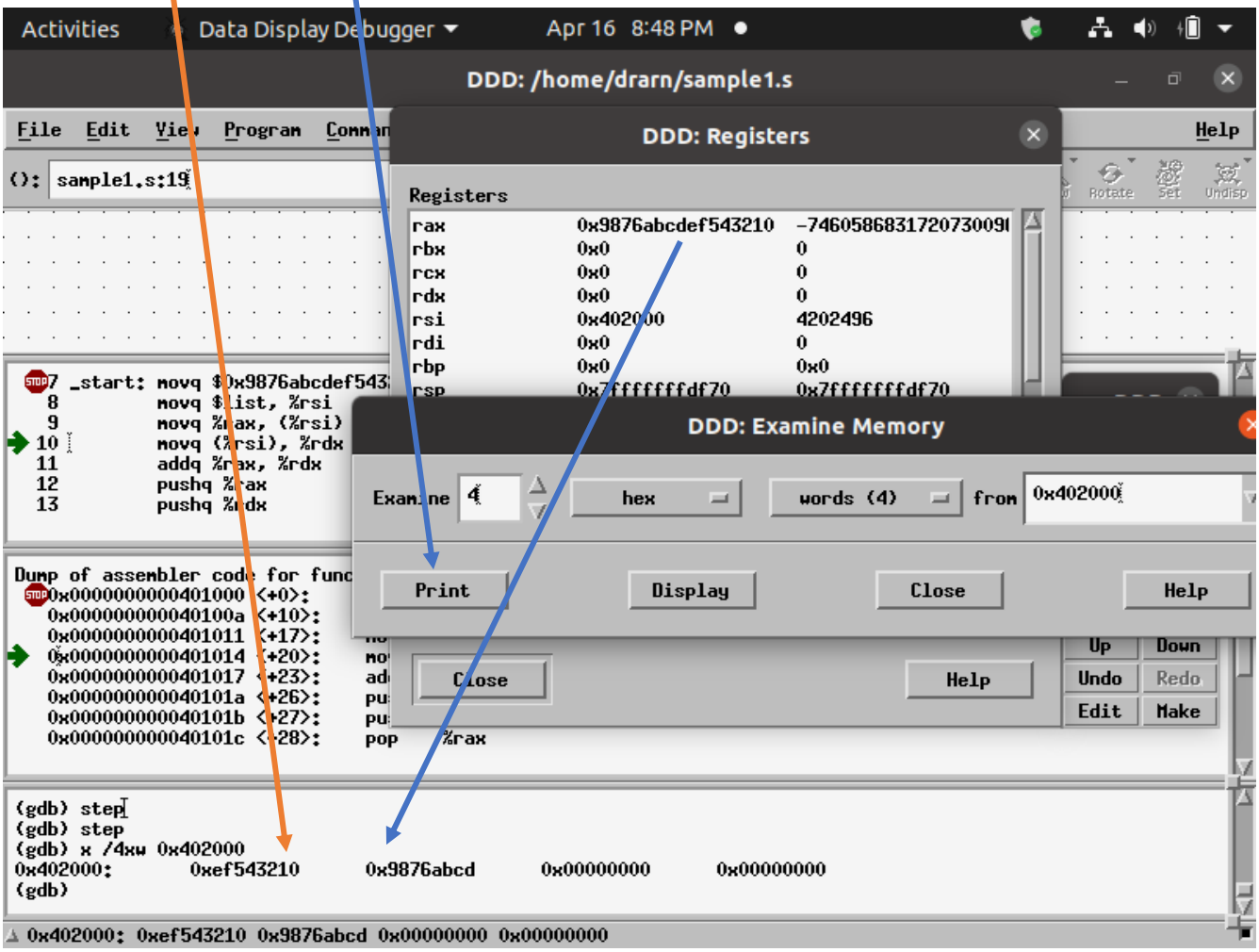
TO DO THIS, CLICK ON Data OPTION IN MENU AND SELECT SUB OPTION Memory UNDER THAT SUB MENU
NOW Examine Memory WINDOW WILL BE POPPED UP



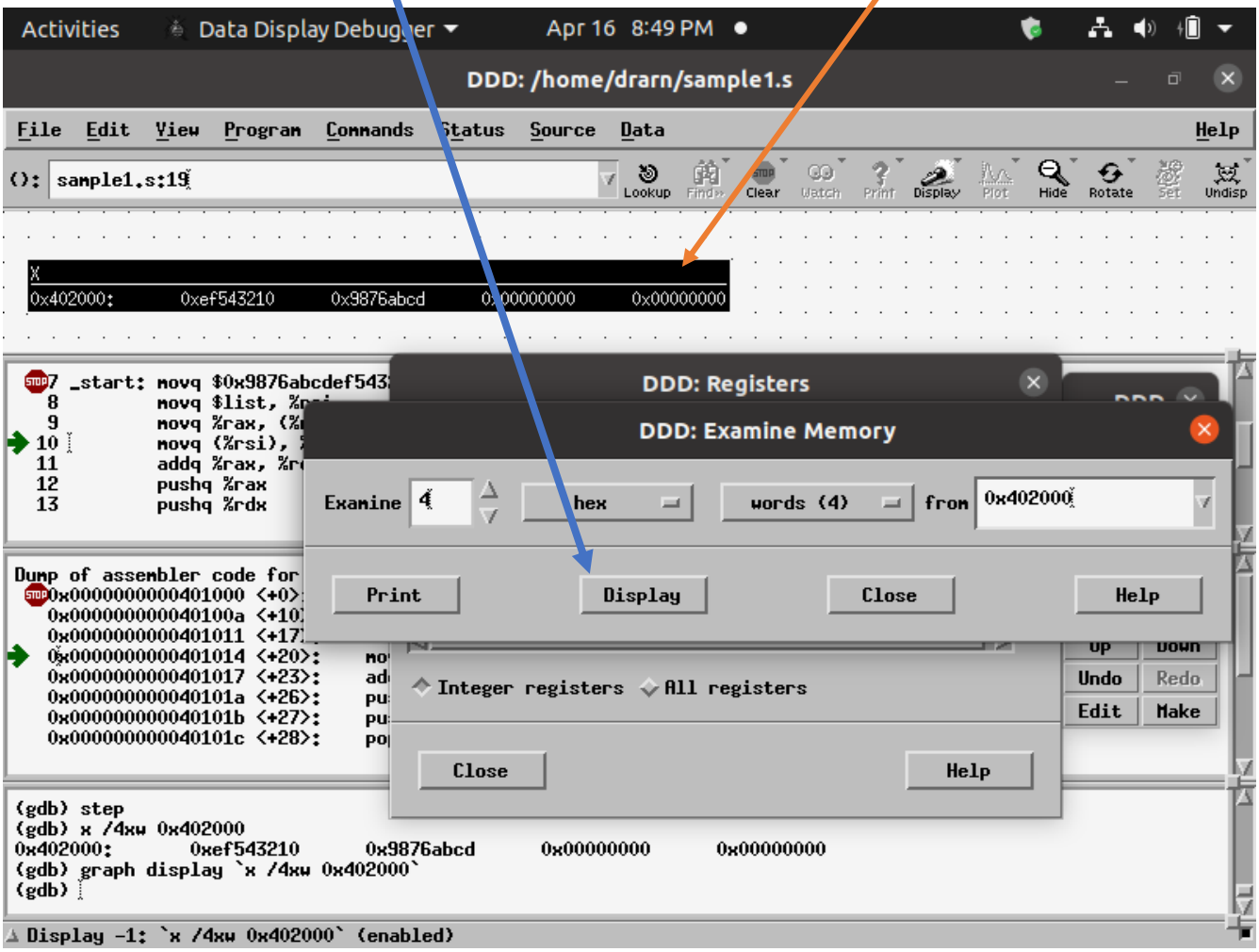
TO EXAMINE THE CONTENTS OF MEMORY STARTING FROM 0x402000,

1. **ENTER STARTING ADDRESS 0x402000 in from text box as shown above**
2. **DATA TO BE DISPLAYED TOGETHER EITHER IN BYTE OR HALFWORD(2 BYTES) OR WORD(4 BYTES) FORMAT – SELECT words(4) as shown above**
3. **DATA display FORMAT – decimal, Octal, hex - SELECT hex as shown above**
4. **NUMBER OF WORDS TO BE EXAMINED STARTING FROM ADDRESS 0x402000 – ENTER 4 IN THE TEXT BOX AFTER Examine TEXT BOX as shown above**

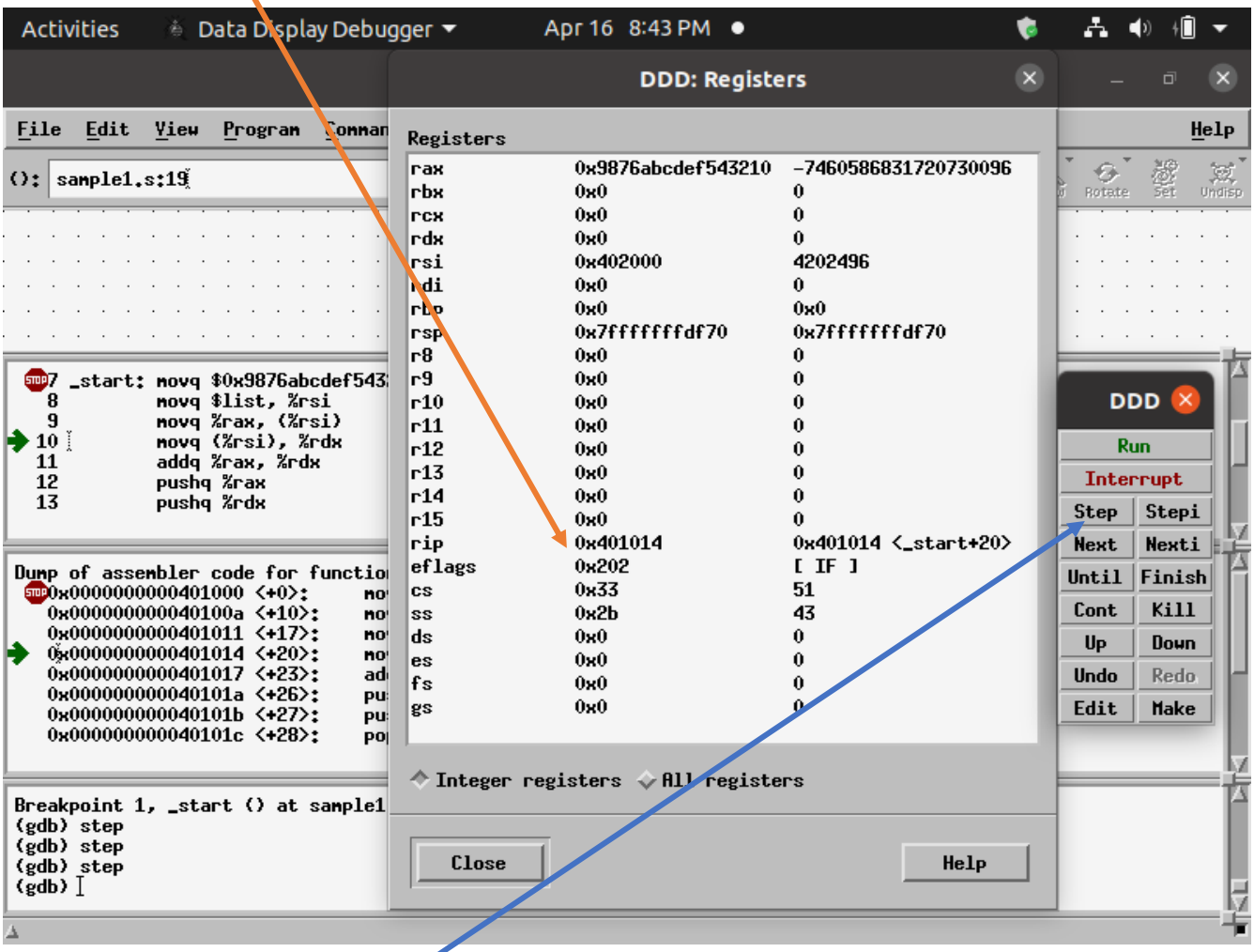
NOW CLICK Print BUTTON TO PRINT 4 WORD MEMORY CONTENTS STARTING FROM ADDRESS 0x402000 IN GDB CONSOLE WINDOW AS SHOWN BELOW -



NOW CLICK Display BUTTON TO PRINT 4 WORD MEMORY CONTENTS STARTING FROM ADDRESS 0x402000 IN DATA WINDOW AS SHOWN BELOW -



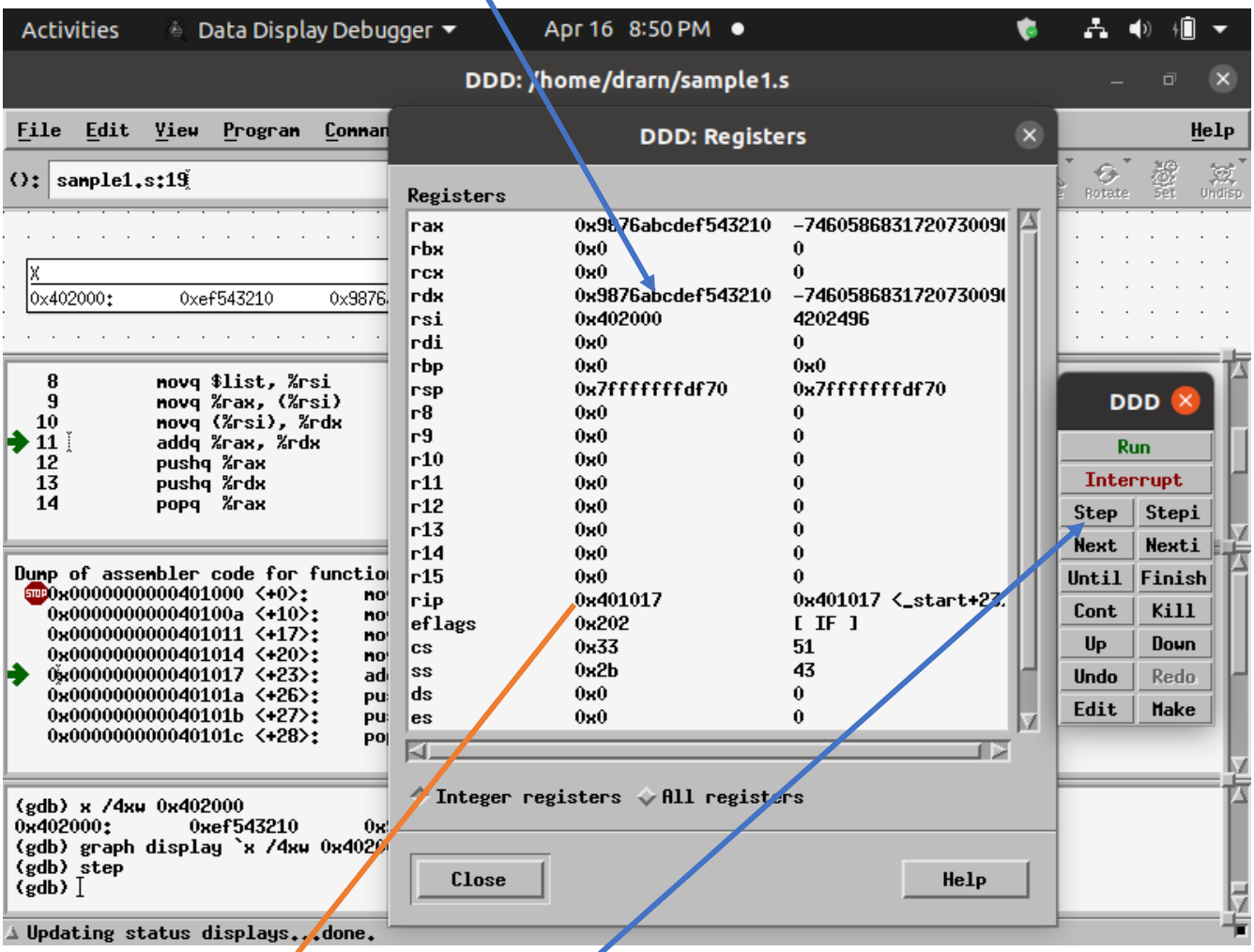
%rip ← 0x401014 ; ADDRESS OF FOURTH INSTRUCTION



NOW CLICK step to execute FOURTH Instruction at line no. 10
movq (%rsi), %rdx

LOOK AT REGISTER WINDOW AFTER THE EXECUTION OF FOURTH INSTRUCTION **movq (%rsi), %rdx**

8 BYTES FROM MEMORY STARTING FROM 0x402000 POINTED TO BY %rsi ARE MOVED TO REGISTER %rdx

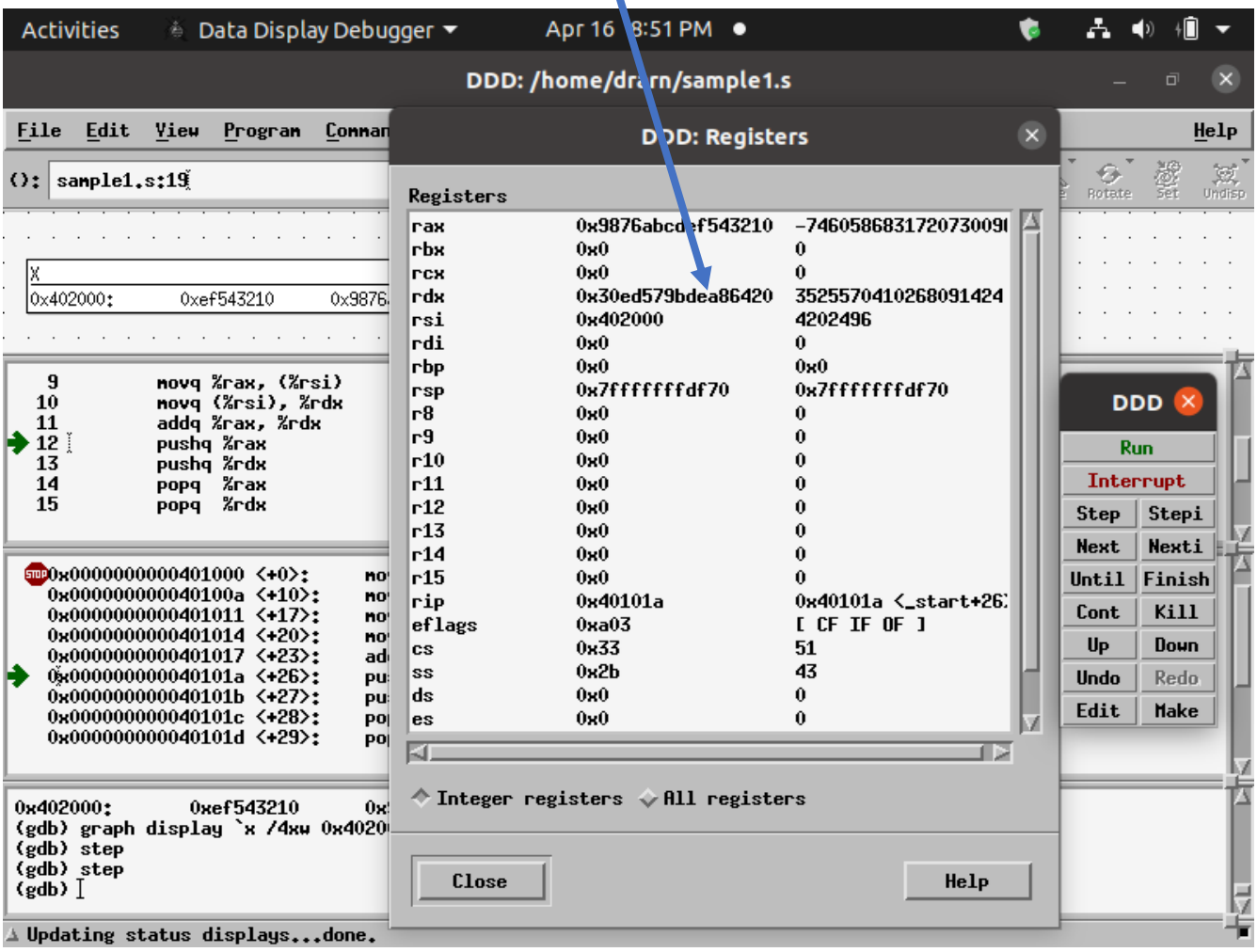


%rip ← 0x401017 ; ADDRESS OF FIFTH INSTRUCTION

NOW CLICK step to execute FIFTH Instruction at line no. 11
addq %rax, %rdx

LOOK AT REGISTER WINDOW AFTER THE EXECUTION OF FIFTH INSTRUCTION addq %rax, %rdx

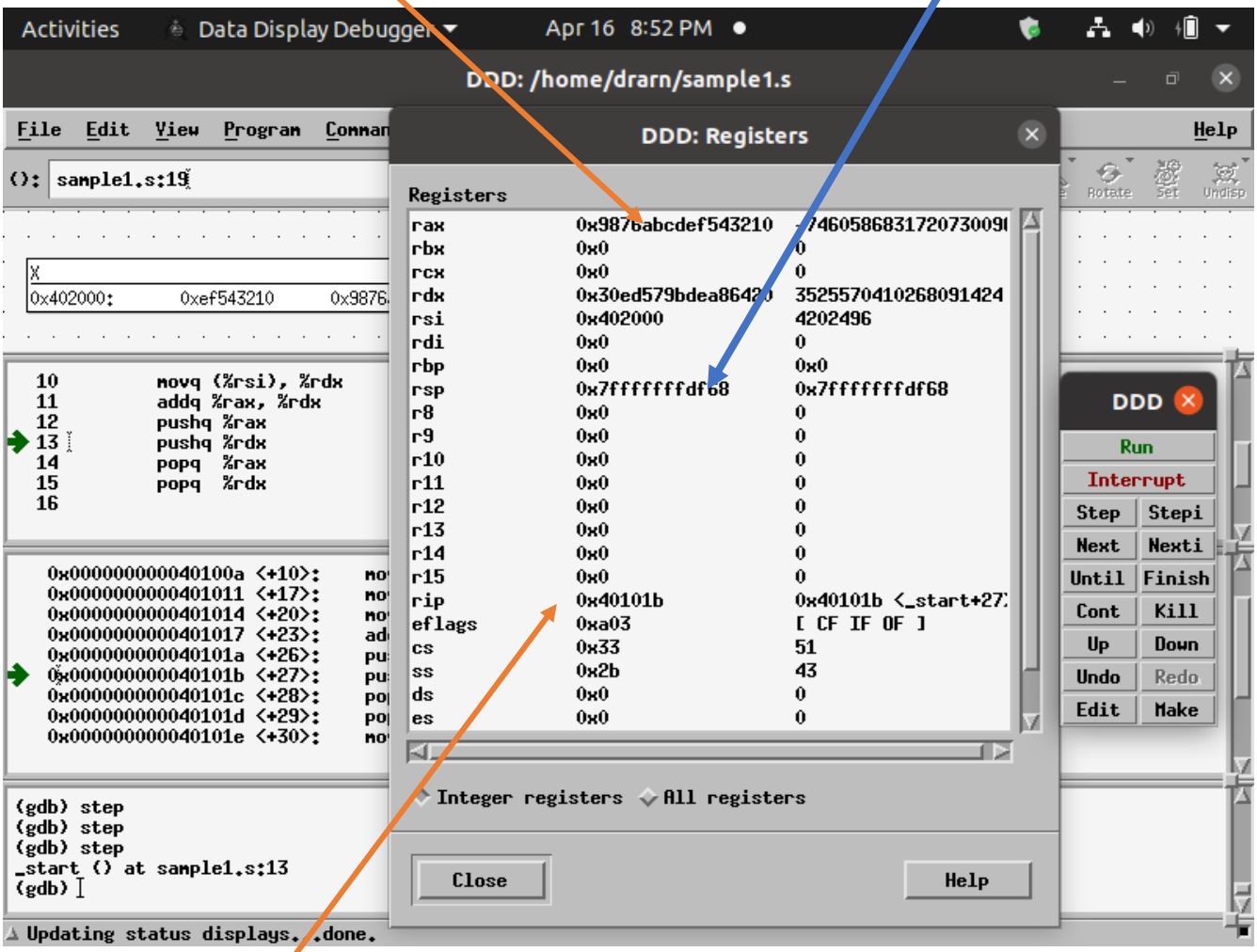
LOOK AT CONTENTS OF REGISTER %rdx



%rip ← 0x40101a ; ADDRESS OF SIXTH INSTRUCTION

NOW CLICK step to execute SIXTH Instruction at line no. 12 pushq %rax

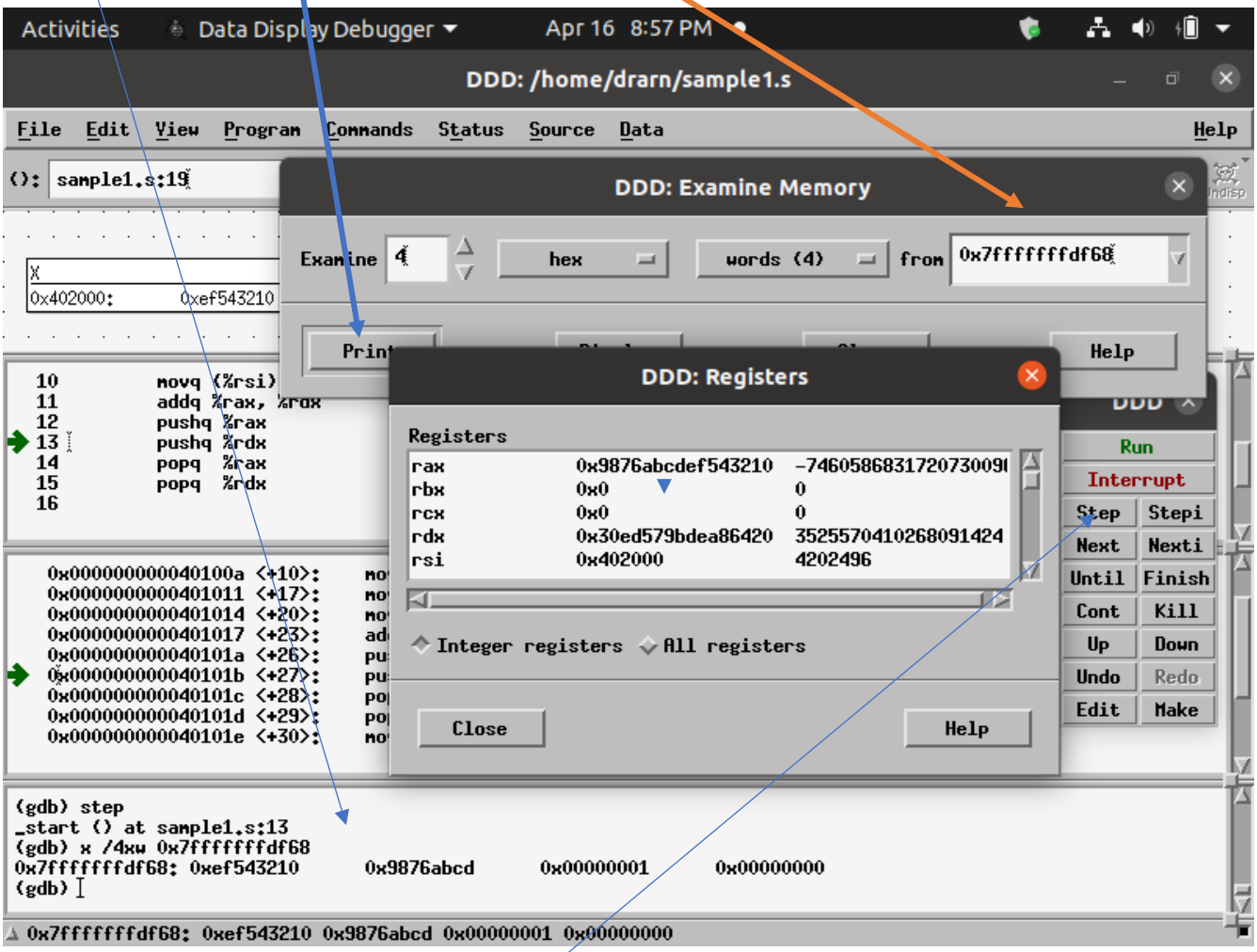
%rsp is decremented by 8 **%rsp = 0x7fffffffdf68**
Contents of %rax are written onto the stack



%rip ← 0x40101b ; ADDRESS OF SEVENTH INSTRUCTION **pushq %rdx**

LET US NOW LOOK AT THE CONTENTS OF THE STACK AT 0x7fffffffdf68

NOW CLICK Print BUTTON TO PRINT 4 WORD STACK MEMORY CONTENTS STARTING FROM ADDRESS 0x7fffffffdf68 IN GDB CONSOLE WINDOW AS SHOWN BELOW -



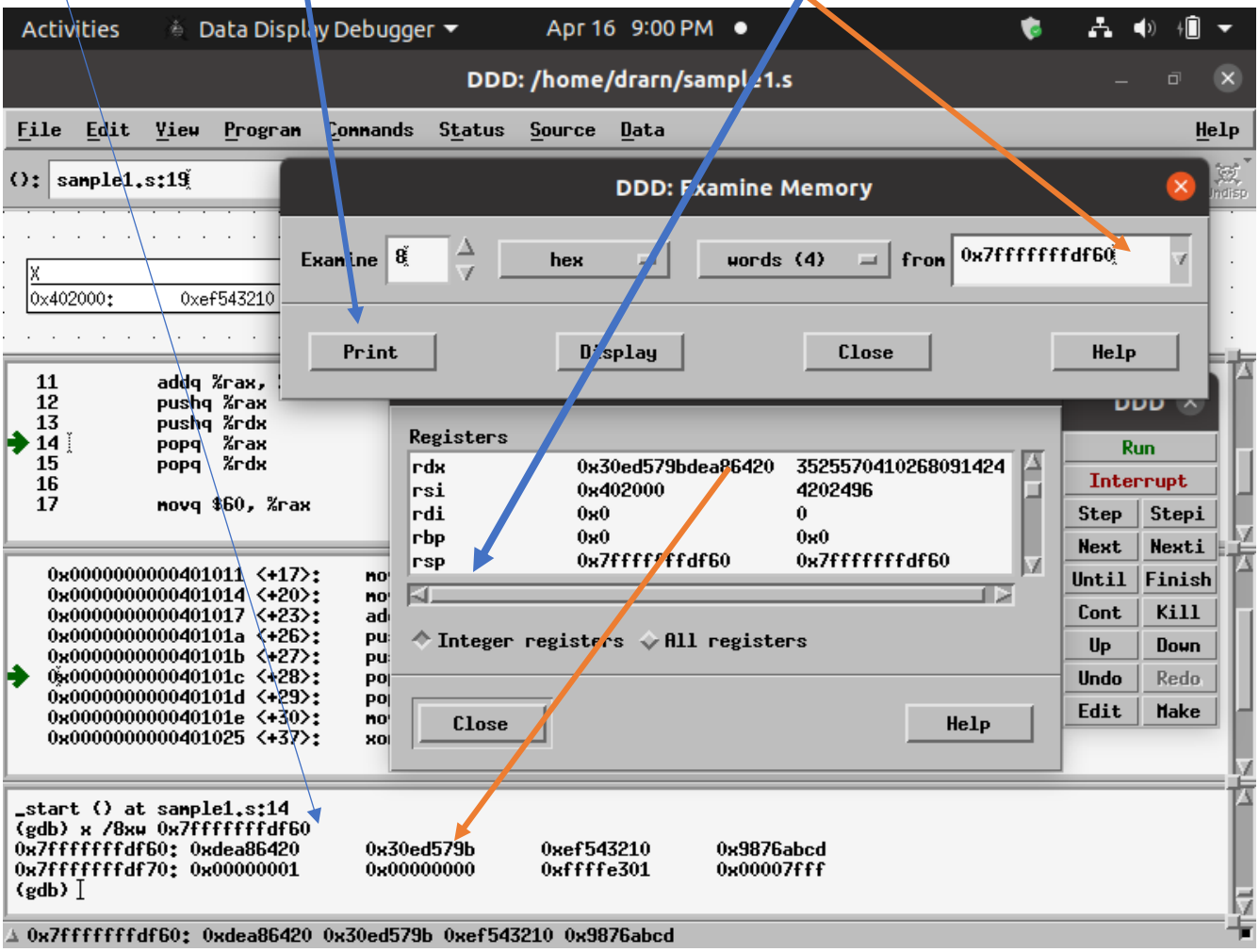
%rip ← 0x40101b ; ADDRESS OF SEVENTH INSTRUCTION

NOW CLICK step to execute SEVENTH Instruction at line no. 13 pushq %rdx

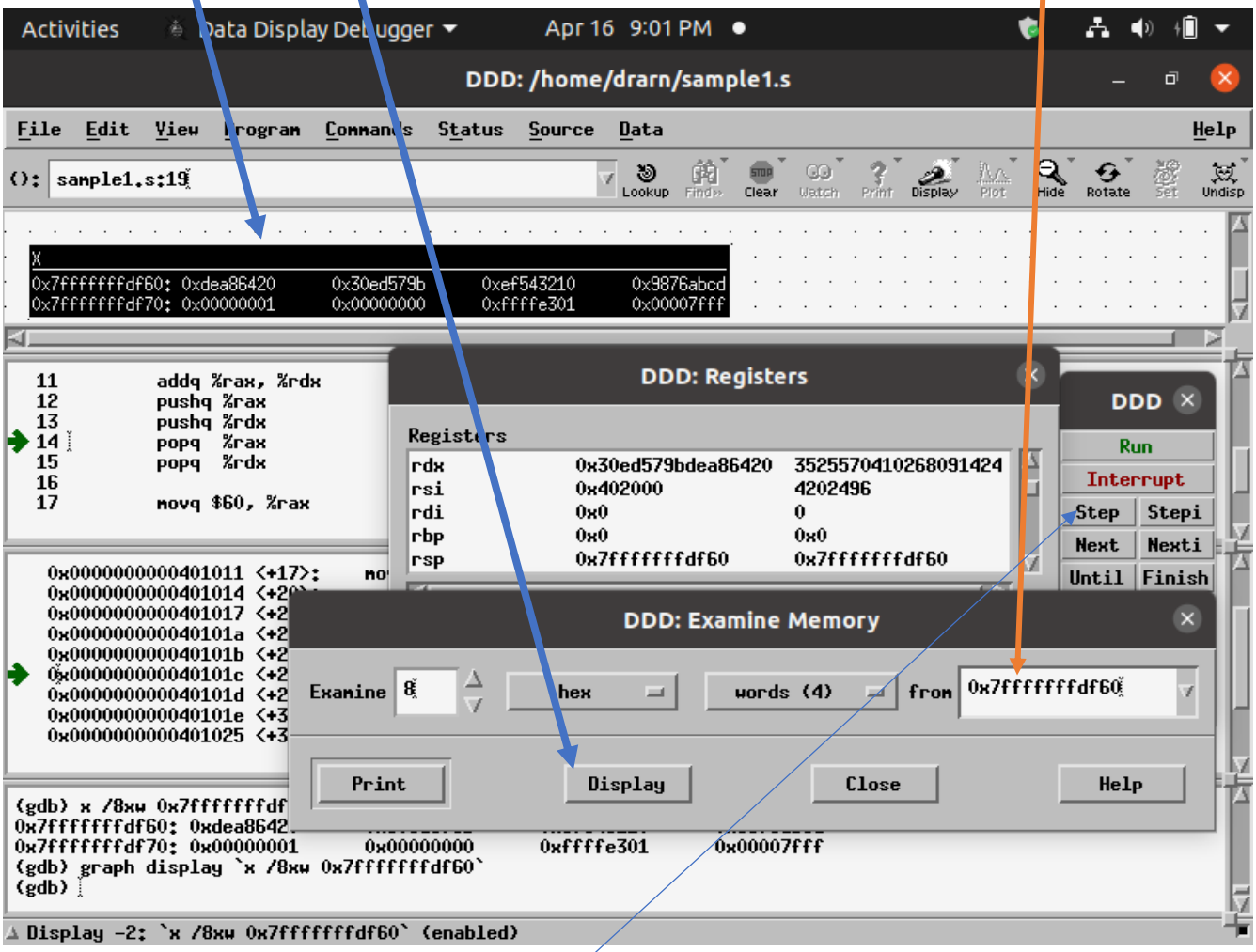
%rsp is decremented by 8 **%rsp = 0x7fffffffdf60**

LET US NOW LOOK AT THE CONTENTS (8 WORDS) OF THE STACK AT 0x7fffffffdf60

NOW CLICK Print BUTTON TO PRINT 8 WORD STACK MEMORY CONTENTS STARTING FROM ADDRESS 0x7fffffffdf60 IN GDB CONSOLE WINDOW AS SHOWN BELOW



NOW CLICK Display BUTTON TO PRINT 8 WORD STACK MEMORY CONTENTS STARTING FROM ADDRESS 0x7fffffffdf60 IN DATA WINDOW AS SHOWN BELOW -

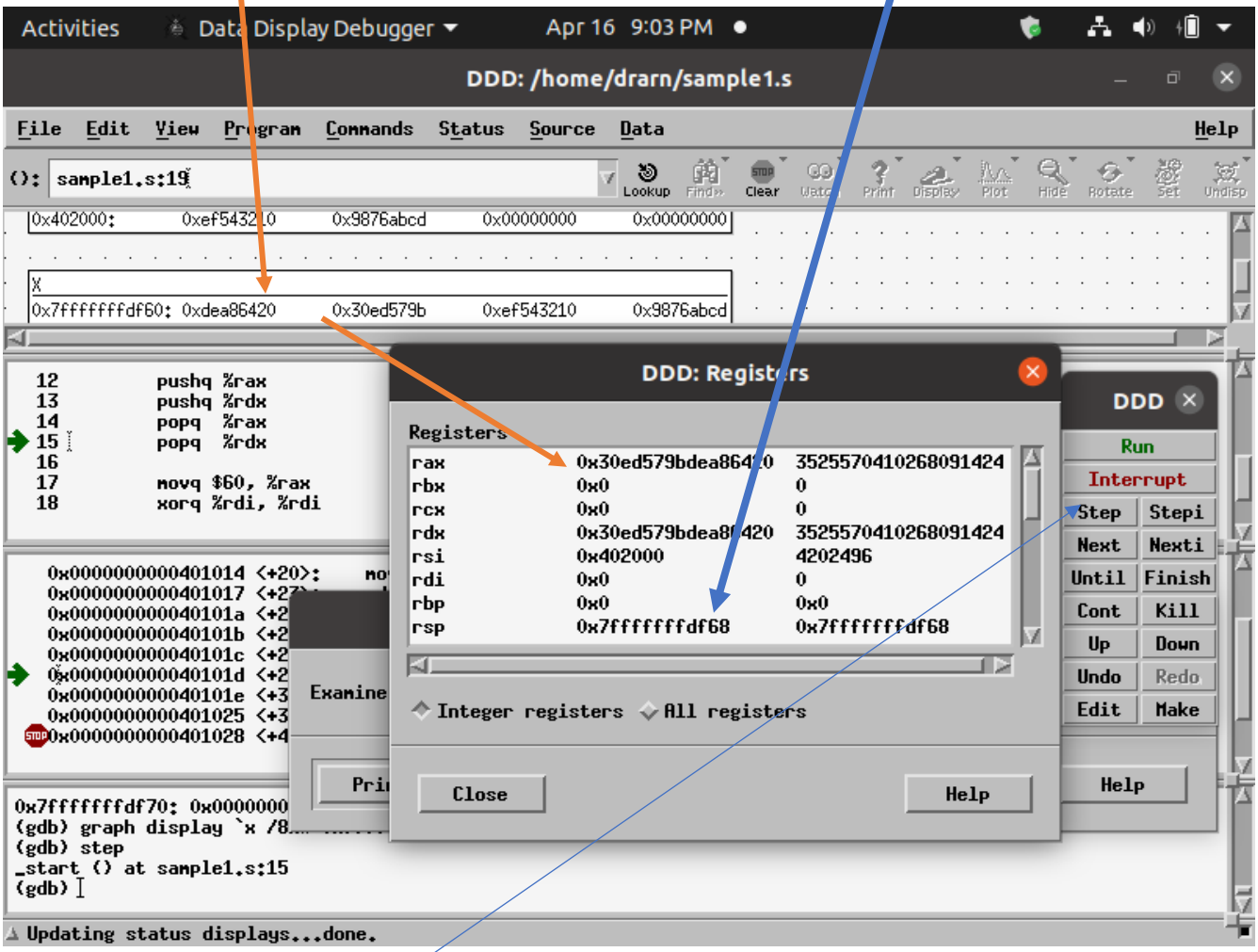


%rip ← 0x40101c ; ADDRESS OF EIGTH INSTRUCTION

NOW CLICK step to execute EIGTH Instruction at line no. 14 popq %rax

Contents (8 bytes) of stack memory area pointed to by %rsp =0x7fffffffdf60 is read into %rax register

After that %rsp is incremented by 8 %rsp = 0x7fffffffdf68

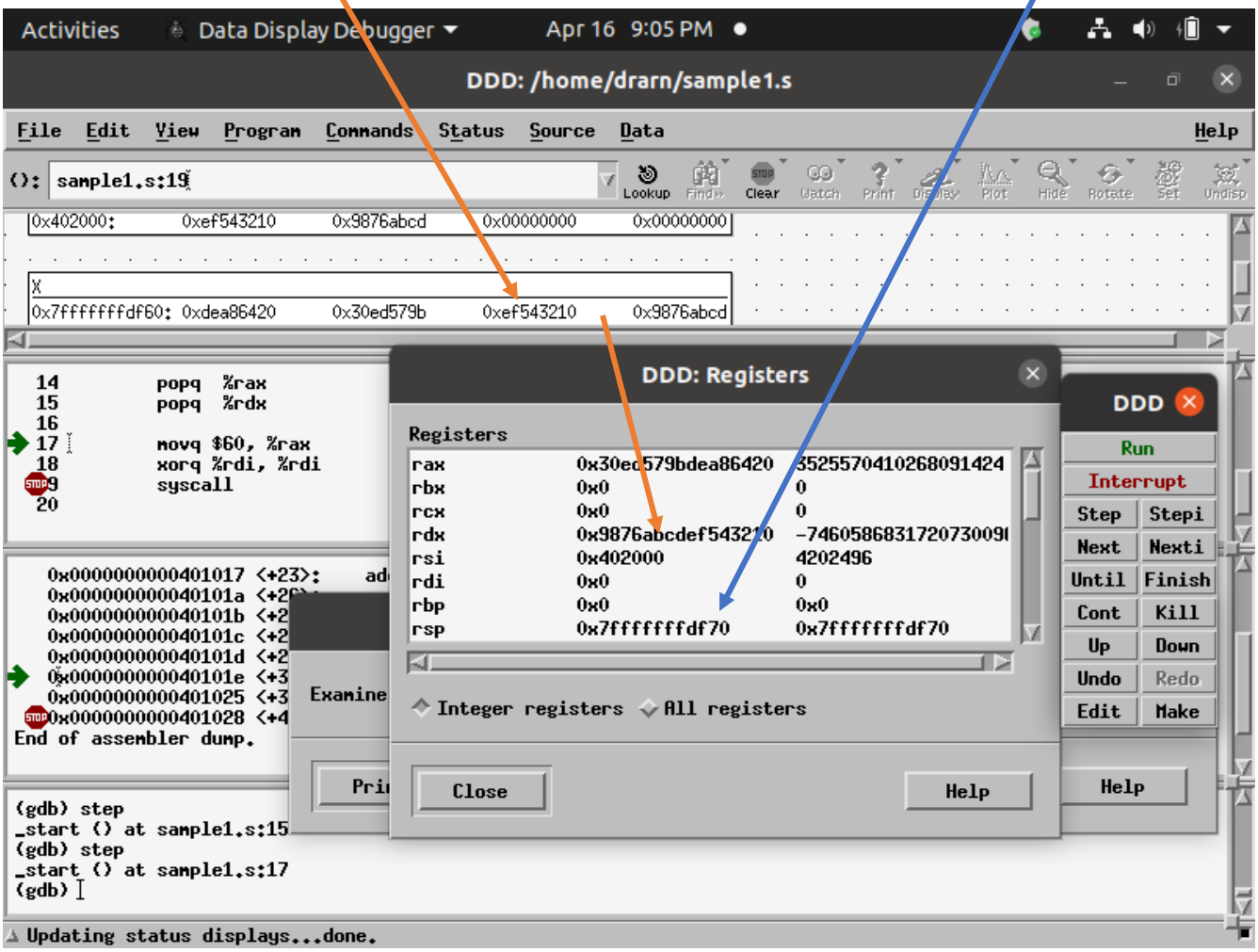


%rip ← 0x40101d ; ADDRESS OF NINTH INSTRUCTION

NOW CLICK step to execute NINTH Instruction at line no. 15
popq %rdx

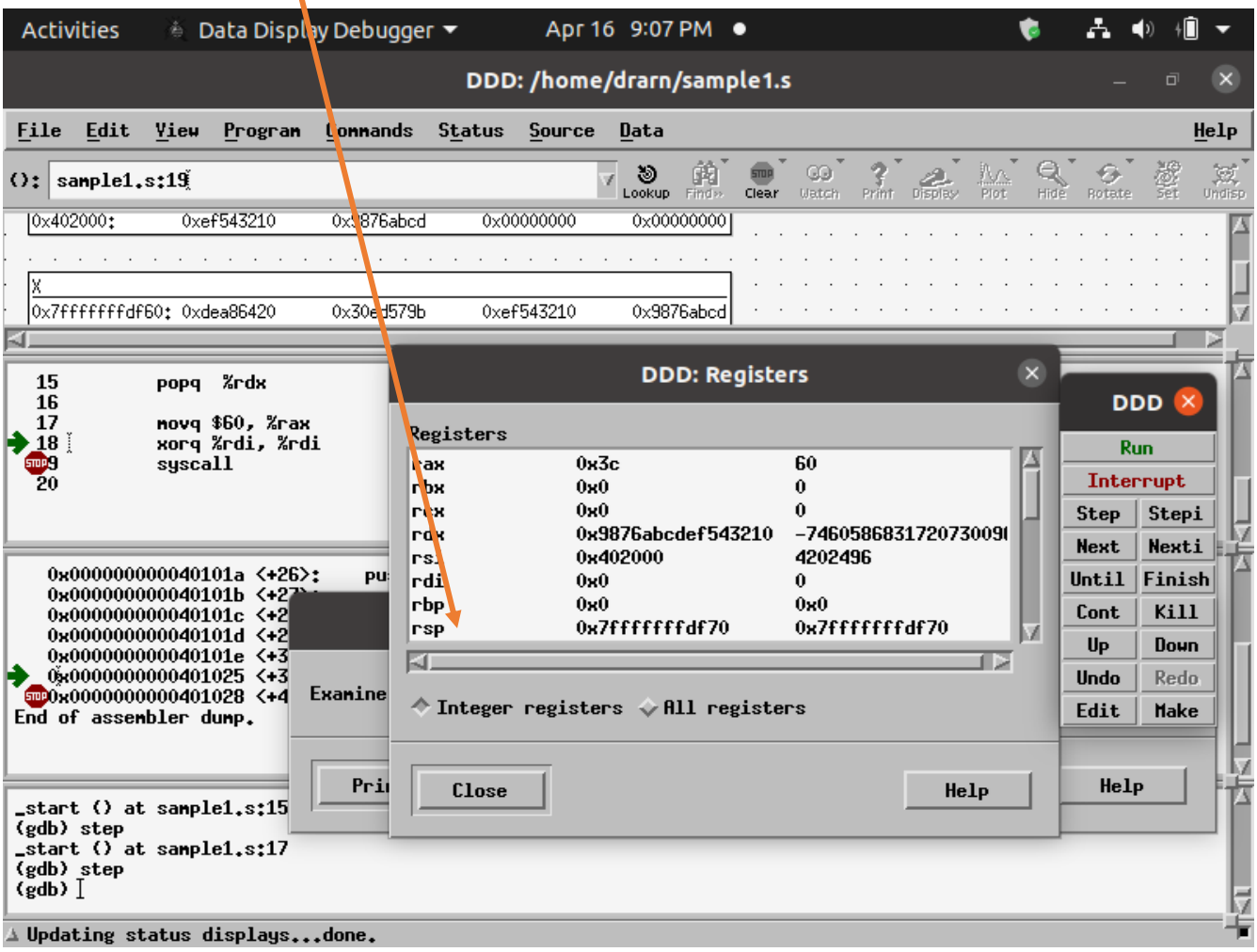
Contents (8 bytes) of stack memory area pointed to by %rsp =0x7fffffffdf68 is read into %rdx register

After that %rsp is incremented by 8 %rsp = 0x7fffffffdf70



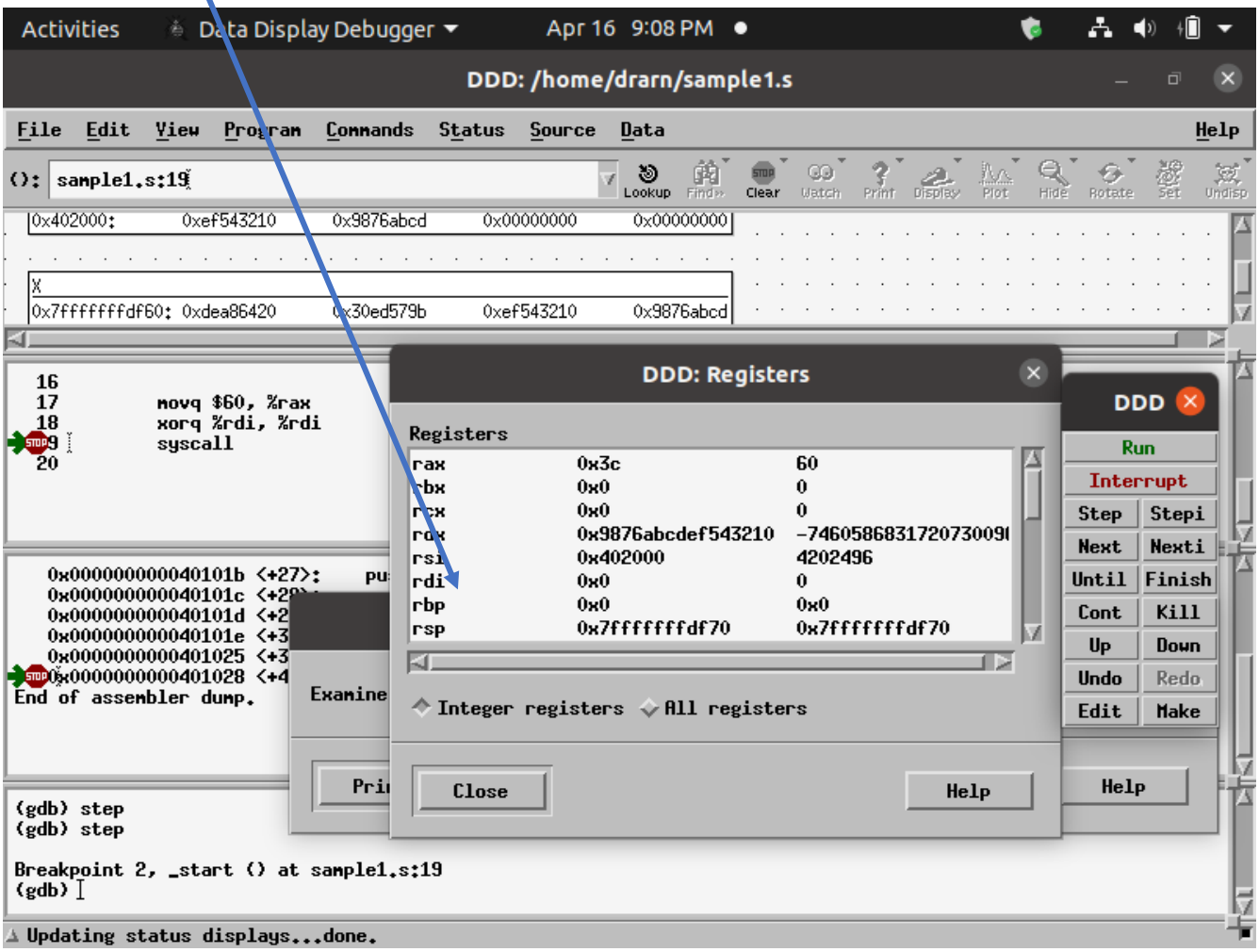
%rip ← 0x40101e ; ADDRESS OF TENTH INSTRUCTION

NOW CLICK step to execute TENTH Instruction at line no. 17
movq $60, %rax



Next %rip ← 0x401025 ; ADDRESS OF ELEVENTH INSTRUCTION

NOW CLICK step to execute ELEVENTH Instruction at line no. 18
xorq %rdi, %rdi

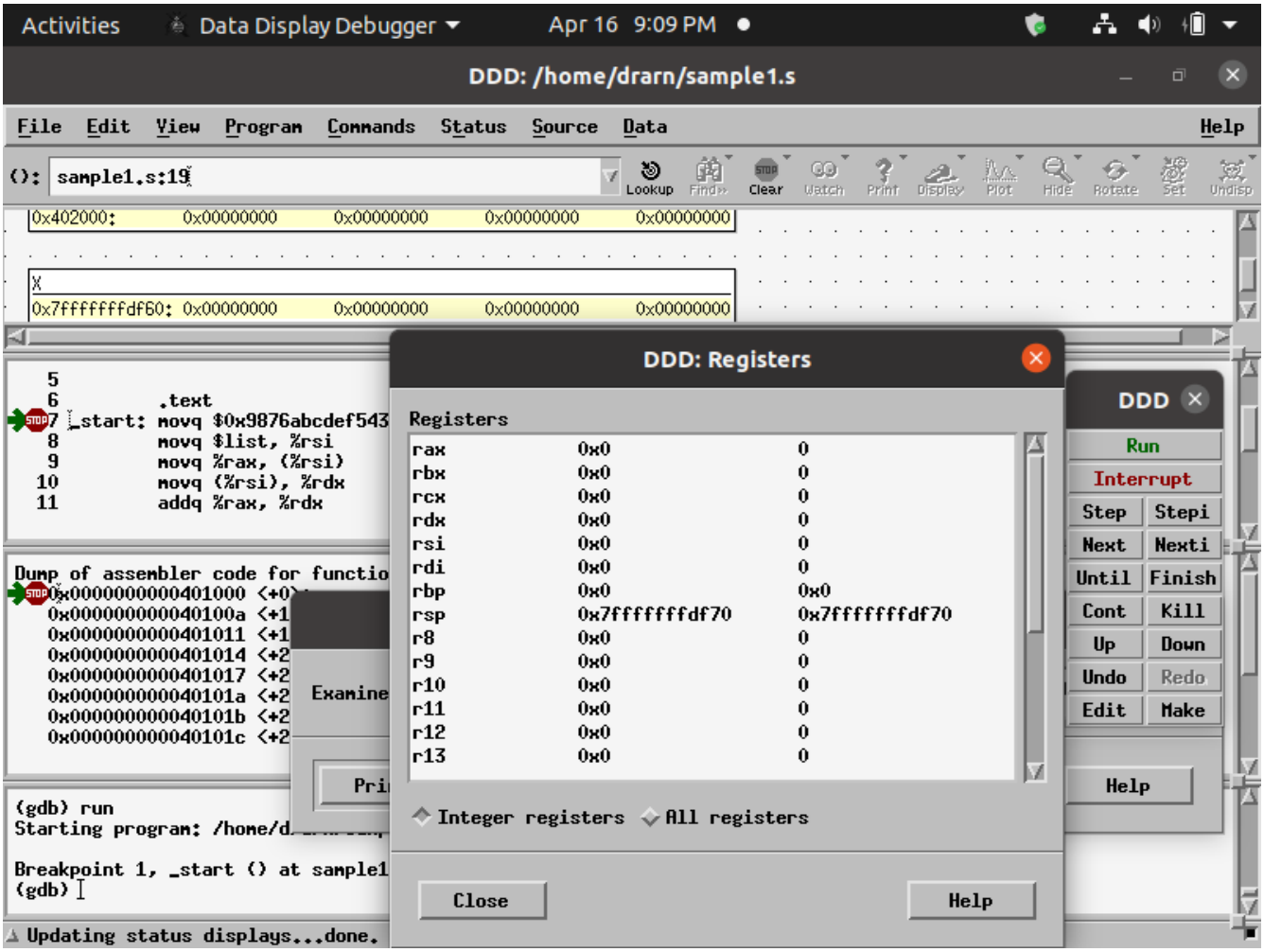


Next %rip ← 0x401028 ; ADDRESS OF TWELVETH INSTRUCTION

NOW CLICK run to execute TWELVETH Instruction at line no. 19
syscall

NOW LOOK AT THE REGISTERS AND MEMORY LOCATIONS AFTER THE EXECUTION OF syscall

ALL ARE INTIALIZED TO THEIR ORIGINAL VALUES AND %rip IS NOW POINTING TO THE FIRST INSTRUCTION



END OF DDD DEBUGGER TUTORIAL SESSION
WISH YOU ALL THE BEST

****